



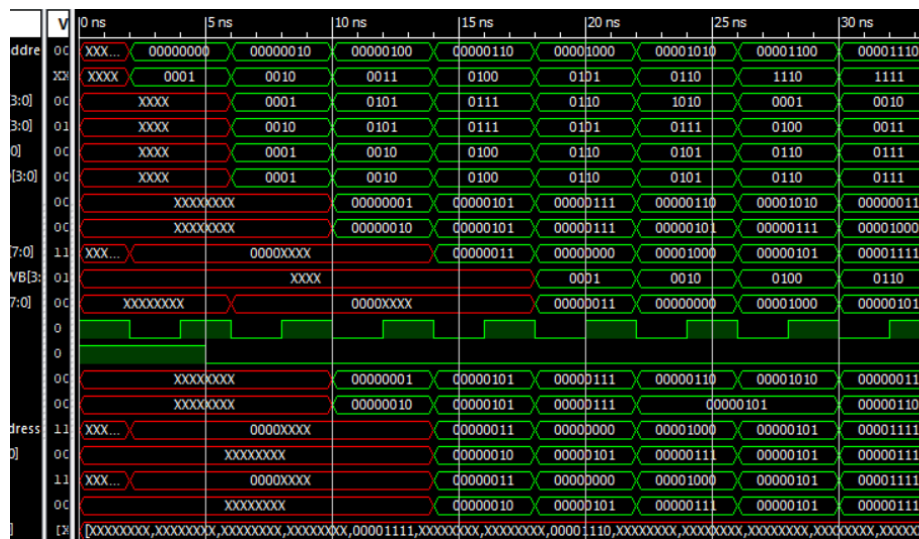
UNIVERSIDAD DE GUADALAJARA
CUCEI



Carrera: Ingeniería en Computación
Materia: Arquitectura de Computadoras

Reporte

PROYECTO FINAL - FASE 2



Integrantes:

Hernández Zúñiga Roberto López López Karla Valeria
Ortiz Segura Emmanuel Alejandro Segura Gutierrez Aaron Ricardo

Maestro: López Arce Delgado Jorge Ernesto

Sección: D03

Fecha: 16 mayo 2026

Índice

1. Introducción

1.1 Procesador MIPS

1.2 Tipos de Instrucciones

1.3 Instrucciones a Implementar

2. Objetivos

2.1 Objetivo General

2.2 Objetivos Específicos

3. Marco Teórico

3.1 Instrucciones Tipo R

3.2 Operador Ternario

3.3 Desglose de Instrucciones Elegidas

3.4 Proceso de Compilación

3.5 Algoritmos Implementables con ISA

3.5 Búsqueda Binaria (Algoritmo)

3.6 Instrucciones Tipo I

3.7 Instrucciones Tipo J

4. Desarrollo

4.1 Pseudocódigo Lenguaje Ensamblador

4.2 Módulos Modificados

4.3 Ensamblador, script Python y .bin

4.4 Resultados Simulación

5. Conclusión

6. Bibliografía

1. Introducción

1.1 Procesador MIPS

En términos sencillos, el procesador MIPS es como un "minimalista" de la tecnología, la esencia del diseño RISC. Su filosofía se basa en que menos es más, al reducir la complejidad de las tareas, el chip puede trabajar a una mayor velocidad.

Puntos clave del diseño

Gestión de Memoria Selectiva:

A diferencia de otros procesadores que se distraen buscando datos en la memoria RAM constantemente, MIPS es muy estricto. Solo usa dos comandos específicos para mover datos (lw-Load word y sw-Store word). Todo el "trabajo sucio" de cálculos ocurre internamente en sus 32 registros, lo que evita cuellos de botella.

Instrucciones Uniformes:

Imaginemos que todas las herramientas en una caja tuvieran exactamente el mismo tamaño y peso. Eso es MIPS, con todas sus instrucciones de 32 bits. Esto permite que el procesador no pierda tiempo midiendo o analizando cada comando; simplemente los recibe y los ejecuta de forma rítmica.

Trabajo en Cadena (Pipelining):

Su estructura está diseñada para funcionar como una línea de ensamblaje industrial. Mientras una instrucción se está terminando de escribir, otra ya se está procesando y una nueva está entrando al procesador. No hay tiempos muertos.

El "Ancla" del Cero:

Tiene una característica curiosa pero brillante: el registro cero. Es un valor constante que nunca cambia, manteniendo siempre un valor de 0, lo que sirve como punto de referencia rápido para inicializar valores, mover y limpiar datos o realizar comparaciones lógicas sin esfuerzo extra.

Las etapas clásicas de un procesador MIPS son:

1. **IF (Instruction Fetch):** Traer la instrucción de memoria.
2. **ID (Instruction Decode):** Decodificar y leer registros.
3. **EX (Execute):** Realizar la operación en la ALU (Unidad Aritmético Lógica).
4. **MEM (Memory Access):** Acceder a la RAM si es necesario.
5. **WB (Write Back):** Escribir el resultado en el registro.

1.2 Tipos de instrucciones

En el diseño de procesadores, existen 3 tipos de instrucciones que facilitan la decodificación en la UC, con formato de 32 bits, se describen de la siguiente forma:

Tipo R (registro) (implementado en esta práctica)

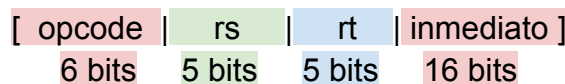
Para las operaciones aritmético-lógicas entre registros, por ejemplo, add y sub. Los 32 bits de la instrucción se dividen de la siguiente manera:



- **rs, rt:** Registros fuente.
- **rd:** Registro destino.
- **funct:** Código funcional que especifica la operación.
- Se decodifican los campos y se envían **rs** y **rt** a la ALU, el resultado de la operación especificada en **function** se escribe en la dirección de memoria **rd**.

Tipo I (inmediato)

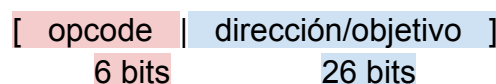
Para operaciones con operandos inmediatos (constantes), cargas/almacenamientos de memoria. Por ejemplo, sw (save word) y lw (load word). Su formato de 32 bits es el siguiente:



- **rs:** Registro fuente.
- **rt:** Registro destino o fuente (depende de la instrucción).
- **inmediato:** Valor constante de 16 bits, a menudo extendido con signo.
- El valor **inmediato** se extiende en signo y se utiliza junto con **rs** en la ALU.

Tipo J (salto/jump)

Para saltos incondicionales a direcciones de memoria. Por ejemplo, j(jump). El formato de instrucción a 32 bits es:



- **dirección:** Junto con el PC (Program Counter), define dirección de salto.

La unidad de control actualiza el PC directamente utilizando la **dirección** de 26 bits.

A continuación, se presenta el set de instrucciones que se utilizará en el proyecto.

1.3 Instrucciones a Implementar

Instrucción	Tipo	Sintaxis	Descripción
add 	R	add \$rd, \$rs, \$rt	Suma: $d = s + t$
sub 	R	sub \$rd, \$rs, \$rt	Resta: $d = s - t$
or 	R	or \$rd, \$rs, \$rt	Operación OR lógica: $d = s t$
and 	R	and \$rd, \$rs, \$rt	Operación AND lógica: $d = s \& t$
slt 	R	slt \$rd, \$rs, \$rt	Si $s < t$, $d = 1$; si no, $d = 0$
nop 	R	nop \$rd, \$rs, \$rt	No Operation: (equivale a sll \$0, \$0, 0)
addi 	I	addi \$rd, \$rs, constante	Suma inmediata: $d = s + \text{constante}$
subi 	I	addi \$rd, \$rs, -constante	Se usa addi con valor negativo
ori 	I	ori \$rd, \$rs, constante	OR inmediato: $d = s \text{constante}$
andi 	I	andi \$rd, \$rs, constante	AND inmediato: $d = s \& \text{constante}$
slti 	I	slti \$rd, \$rs, constante	SLT inmediato: Si $s < \text{constante}$, $d = 1$
lw 	I	lw \$rd, desplazamiento(\$rs)	$rd = rs + \text{desplazamiento}$
sw 	I	sw \$rd, desplazamiento(\$rs)	$rs + \text{desplazamiento} = rd$
beq 	I	beq \$rs, \$rt, etiqueta	Branch if Equal: Salta si $s = t$
bne 	I	bne \$rs, \$rt, etiqueta	Branch if Not Equal: Salta si $s \neq t$
bgtz 	I	bgtz \$rs, etiqueta	Branch if Greater Than Zero: Salta si $s > 0$
j 	J	j etiqueta	Salto incondicional a dirección de etiqueta
xor 	R	xor \$rd, \$rs, \$rt	$d = 1$, si los bits de s y t son diferentes
teq 	R	teq \$rs, \$rt	Excepción si el contenido de $s = t$
tge 	R	tge \$rs, \$rt	Excepción si $s \geq t$
sra 	R	sra \$rd, \$rt, shamt	Desplaza bits de t a la derecha shamt veces, manteniendo el bit de signo
srl 	R	srl \$rd, \$rt, shamt	Desplaza los bits de t a la derecha shamt veces y rellena dependiendo del signo

-  Instrucciones Aritmeticas y Logicas
-  Instrucciones con Inmediatos
-  Instrucciones de Memoria (Carga y Almacenamiento)
-  Instrucciones de Control de Flujo (Saltos)
-  Instrucciones de Desplazamiento
-  Instrucciones de Excepción (Trap)

2. Objetivos

2.1 Objetivo general

Explicar el funcionamiento del Datapath e implementar el diseño, tomando en cuenta todos los módulos necesarios para su funcionamiento, tales como, el banco de registros, ALU, multiplexores, unidades de control, desde la investigación, organización, división de tareas, implementación y ejecución, utilizando para esta primera fase las instrucciones de tipo R especificadas en la tabla descrita dentro de la introducción.

2.1 Objetivos específicos

- Investigar de manera profunda cada concepto para su entendimiento y aplicación.
- Desarrollar el código de prueba en ensamblador para instrucciones tipo R.
- Hacer las modificaciones necesarias a los módulos anteriormente diseñados para ejecutar todas las instrucciones tipo R de la tabla.
- Hacer el script de Python para convertir el archivo de código ensamblador a binario.
- Realizar los Test Bench necesarios para comprobar el funcionamiento de la práctica.

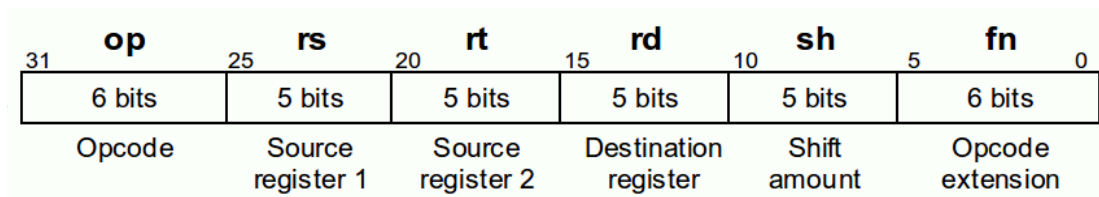
3. Marco Teórico

3.1 Instrucción Tipo R

En la arquitectura MIPS, las instrucciones tipo R (Register-type) se utilizan para realizar operaciones entre registros. Este tipo de instrucción se caracteriza por operar exclusivamente con datos almacenados en registros internos del procesador, sin acceso directo a la memoria.

El formato (máquina) de instrucción tipo R se compone de los siguientes campos:

- **opcode:** código de operación (generalmente 0 en tipo R)
- **rs:** primer registro fuente
- **rt:** segundo registro fuente
- **rd:** registro destino
- **shamt:** desplazamiento (shift amount)
- **funct:** especifica la operación exacta



Representación de la separación de bits para cada campo.

Como se observa en la imagen, de los 32 bits provenientes de la instrucción el campo function y el opcode necesitan de 6 bits para llevar a cabo la operación, y los campos restantes constan de 5 bits.

De acuerdo a esta información, la separación de campos en verilog para instrucciones del tipo R en formato MIPS, según los bits especificados para cada apartado, quedaría algo así:

```
assign opcode = instruccion[31:26];
assign rs     = instruccion[25:21];
assign rt     = instruccion[20:16];
assign rd     = instruccion[15:11];
assign shamt  = instruccion[10:6];
assign funct  = instruccion[5:0];
```

Opcode y **funct** con 6 bits cada uno, **rs**, **rt**, **rd** y **shamt** con 5 bits, dando el total de 32 bits para la instrucción.

Sintaxis para Ensamblador

La sintaxis básica para instrucciones tipo R (Registro-Registro) en lenguaje ensamblador, es la siguiente:

`opcode registro_destino , registro_fuente1 , registro_fuente2`

Ejemplo:

`add $7 , $4 , $2`

Ejemplo de la sintaxis para una suma en código ensamblador, se usa el operador `add`. Al número en la dirección de memoria 4 se le suma el número en la dirección de memoria 2, el resultado se escribe en la dirección de memoria 7.

Elementos de la Sintaxis

- **Mnemónico (Opcode):** Nombre de la instrucción (ej. ADD, SUB, AND, OR, SLT).
- **Registro Destino (rd):** El primer operando tras el mnemónico, donde se almacenará el resultado.
- **Registro Fuente 1 (rs):** Segundo operando, primer valor de entrada a la ALU.
- **Registro Fuente 2 (rt):** Tercer operando, segundo valor de entrada a la ALU.

Operaciones implementables

Existe una gran cantidad de operaciones aritmético-lógicas a realizar con las instrucciones tipo R, tales como `add`(suma), `sub`(resta), `and`, `or`. Dentro de estas una de las operaciones más relevantes es **SLT** (Set on Less Than). Esta instrucción compara dos registros y establece el valor del registro destino dependiendo del resultado de la comparación:

- Si **rs < rt**, entonces **rd = 1**
- En caso contrario, **rd = 0**

La instrucción SLT es fundamental para la implementación de estructuras de control en bajo nivel, como condicionales (`if`) y ciclos (`while`), ya que permite realizar comparaciones entre valores.

Internamente la ALU no puede realizar la comparativa entre un valor y otro e indicar si es menor o mayor directamente, es por eso que, esta operación suele implementarse mediante una resta (**rs - rt**) y la evaluación del bit de signo del resultado.

3.2 Operador Ternario

La operación ternaria es común en lenguajes de programación de alto nivel como C y Verilog. Se trata de una forma compacta de expresar una estructura condicional.

La sintaxis de este operador en C sería la siguiente:

condición ? valor_si_verdadero : valor_si_falso;

Esta operación evalúa una condición lógica. Si la condición es verdadera, retorna el primer valor, si es falsa, retorna el segundo.

En el contexto de diseño digital (por ejemplo, en Verilog), la operación ternaria se utiliza frecuentemente para describir lógica combinacional. Por ejemplo:

assign salida = (a < b) ? 1 : 0;

Aquí se observa una equivalencia directa con la instrucción SLT, ya que se está evaluando una condición de comparación y asignando un valor binario como resultado. Es útil, ya que simplifica el código y puede representar directamente multiplexores en hardware, lo que facilita la síntesis.

3.3 Desglose de Instrucciones Elegidas

Las instrucciones xor, sra y srl, son lógicas y de desplazamiento, estas instrucciones operan a nivel de bits o desplazan la posición de los mismos dentro de un registro.

- **XOR** (lógica): Operación or exclusiva bit a bit entre los valores de 2 registros y almacena el resultado en un tercer registro.
- **SRL** (lógica): Siempre rellena con 0.
- **SRA** (aritmética): Rellena con el valor del bit de signo (si el número era negativo, rellena con 1: si era positivo, con 0) para preservar el valor matemático.

Las instrucciones como teq y tge, de tipo R, comparan dos registros. Si la condición se cumple, el procesador genera una excepción (interrupción de software), lo que suele detener el programa o saltar a un manejador de errores.

- **TEQ** (excepción): Compara dos registros y genera una excepción (trampa) si son iguales, se utiliza para gestionar errores de software, como la división por cero, terminando el programa y transfiriendo el control al gestor de excepciones.
- **TGE** (excepción): Genera una trampa si $rs \geq rt$. Se usa comúnmente para verificar límites, depuración, validación de índices de arreglos.

3.4 Fases del Proceso de Compilación

Este es el proceso que transforma un programa escrito en un lenguaje de programación de alto nivel (código fuente) a un lenguaje de bajo nivel (código objeto o binario). A diferencia de un intérprete (que traduce línea por línea mientras el programa corre), el compilador traduce todo el programa antes de que se ejecute.

El transcurso desde que presionamos “compilar” hasta que obtienes un archivo .exe o un binario para MIPS, como será en esta práctica, se divide en etapas o fases:

- ❖ **Análisis Léxico (Scanner)**

El compilador lee el código fuente carácter por carácter y los agrupa en unidades con significado llamadas *tokens*. Por ejemplo: Si escribes `suma = 5 + 10`, el analizador identifica: un identificador (suma), un operador de asignación (=), un número (5), etc.

- ❖ **Análisis Sintáctico (Parser)**

Aquí se revisa que los *tokens* sigan las reglas gramaticales del lenguaje. Se crea una estructura jerárquica llamada Árbol de Sintaxis Abstracta (AST). Si te falta un punto y coma o un paréntesis, aquí es donde el compilador te lanza el error.

- ❖ **Análisis Semántico**

El compilador verifica que el código tenga sentido, por ejemplo, al sumar una palabra con un número, o usar una variable no declarada, si es así, el proceso se detiene.

- ❖ **Generación de Código Intermedio**

El compilador crea una versión del código que es independiente de la máquina. Es una representación simplificada que facilita las optimizaciones antes de pasar al lenguaje específico de un procesador.

- ❖ **Optimización de Código**

Esta es la fase "inteligente". El compilador intenta que el programa sea más rápido o ocupe menos memoria. Por ejemplo, si tienes una operación como `x = 2 * 4`, el optimizador la cambia directamente por `x = 8` para ahorrarle trabajo al procesador.

- ❖ **Generación de Código de Salida**

El compilador traduce el código optimizado al lenguaje específico de la arquitectura destino (como las instrucciones de 32 bits de MIPS o de x86).

¿Qué pasa cuando se completaron las fases?

El compilador, no es el proceso completo. Después de compilar, se obtiene un “archivo objeto” (.o o .obj), pero este aun no funciona solo.

El **Linker** o enlazador toma el archivo objeto y lo une con las librerías del sistema (por ejemplo, las que permiten imprimir en pantalla o leer el teclado) para generar el ejecutable final.

Entonces se inicia por un **análisis**, entender que escribió el programador y detectar errores de escritura, pasa por una **síntesis**, construir el programa en lenguaje máquina y optimizarlo, y finalmente llegamos al **enlazado**, unir todas las piezas y librerías en un solo archivo funcional.

3.5 Algoritmos Implementables con la ISA

De acuerdo al set de instrucciones definido para este proyecto, se pueden llegar a construir algoritmos con cierta complejidad lógica y aritmética, combinando instrucciones de registro, inmediatas y saltos.

Estructuras de control de flujo (lógica condicional)

Con instrucciones de salto y comparación, podemos recrear cualquier lógica de decisión:

- **Sentencias if-else:** Utilizando beq y bne para saltar a diferentes bloques de código según si dos registros son iguales o no.
- **Bucles (for, while):** Combinando saltos condicionales (bgtz, beq) con saltos incondicionales (j) al final del bloque para repetir una secuencia.
- **Validaciones críticas:** Usando las instrucciones de excepción teq y tge para detener el programa si ocurre un error lógico (como un índice fuera de rango o una condición prohibida).

Manipulación de Estructuras de Datos

Con las instrucciones de carga y almacenamiento, puedes gestionar datos en la memoria RAM:

- **Recorrido de Arreglos (Arrays):** Utilizando lw (cargar) y sw (guardar) junto con addi para incrementar el puntero o índice de memoria.
- **Copia de datos:** Algoritmos para mover bloques de información de una dirección de memoria a otra.

Aritmética y Procesamiento de Bits

- **Cálculos Matemáticos:** Implementación de fórmulas complejas usando add, sub y addi.
- **Multiplicación y División por potencias de 2:** Usando srl y sra para desplazar bits, lo cual es mucho más rápido que realizar operaciones matemáticas completas.
- **Máscaras de Bits:** Con and, or, xor y andi, puedes extraer, encender o invertir bits específicos dentro de una palabra de datos (útil para protocolos de comunicación o control de hardware).

Algoritmos de Ordenamiento y Búsqueda

Es posible implementar algoritmos clásicos de ciencias de la computación:

- **Búsqueda Lineal:** Recorriendo la lista con lw y comparando con beq.
- **Ordenamiento** (como el método de la burbuja): Utilizando slt o slti para comparar valores y sw para intercambiar sus posiciones en memoria si uno es menor que otro.

3.6 Búsqueda Binaria (Algoritmo) Aún no se implementa en esta fase

La búsqueda binaria es uno de los algoritmos de búsqueda más eficientes en informática, el método que utiliza es sencillo, ya que, para encontrar algo, no necesitas mirar cada elemento, solo necesitas saber hacia qué dirección moverte.

El algoritmo utiliza una estrategia de división sucesiva. En lugar de revisar uno por uno (como la búsqueda secuencial), este método divide el espacio de búsqueda a la mitad en cada paso. Para que la búsqueda binaria funcione, la colección de datos (arreglo o lista) debe estar ordenada (ya sea de forma ascendente o descendente). Si los datos están desordenados, el algoritmo fallará, ya que no podrá descartar mitades con seguridad.

Funcionamiento

Imagina que buscas un número X en un arreglo:

- **Definir límites:** Se establecen dos punteros: *Inicio* (posición 0) y *Fin* (última posición).
- **Calcular el punto medio:** Se halla el índice central usando la fórmula:

$$\text{Medio} = \frac{\text{Inicio} + \text{Fin}}{2}$$

- **Comparar:**

- Si el elemento en *Medio* es igual a *X*: La búsqueda termina.
- Si el elemento en *Medio* es mayor que *X*: El número debe estar en la mitad izquierda. Entonces, movemos el puntero *Fin* a *Medio* - 1.
- Si el elemento en *Medio* es menor que *X*: El número debe estar en la mitad derecha. Entonces, movemos el puntero *Inicio* a *Medio* + 1.
- **Repetir:** El proceso vuelve al paso 2 con los nuevos límites hasta encontrar el valor o hasta que *Inicio* supere a *Fin* (lo que significa que el valor no existe en la lista).

Lo que hace especial a este algoritmo es su velocidad. Mientras que una búsqueda lineal tarda un tiempo proporcional al número de elementos (*n*), la búsqueda binaria reduce el problema exponencialmente.

Ventajas	Desventajas
Búsqueda rápida en listas grandes.	Requiere datos pre-ordenados.
Consumo de recursos mínimo.	Menos eficiente en listas pequeñas.
Fácil de implementar recursivamente.	Insertar datos obliga a reordenar.

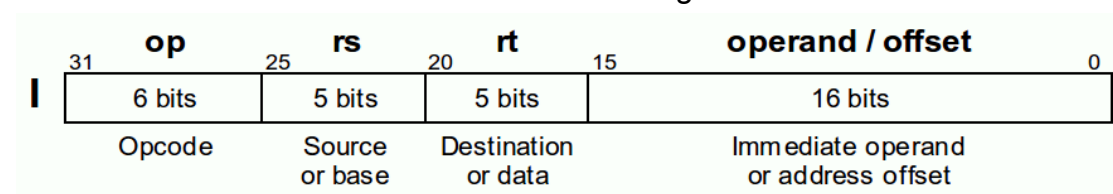
3.7 Instrucciones Tipo I:

Las instrucciones Tipo I (Immediate-type) son fundamentales en la arquitectura MIPS para realizar operaciones que involucran operandos constantes (inmediatos) y para gestionar la transferencia de datos entre los registros del procesador y la memoria principal (RAM) . A diferencia de las instrucciones puramente aritméticas que operan sobre datos preexistentes en registros, el formato Tipo I dota al procesador de la capacidad de introducir nuevos valores al sistema y de acceder a estructuras de datos masivas guardadas en memoria.

El formato Tipo I mantiene la longitud estándar de 32 bits, pero redistribuye sus campos para albergar un número constante. Los campos se dividen de la siguiente manera:

- Opcode (Código de Operación): 6 bits que definen la instrucción exacta (ej. lw, sw, addi).
- rs (Register Source): 5 bits que indican el primer registro fuente. En operaciones de memoria, este registro contiene la "dirección base".
- rt (Register Target): 5 bits que indican el registro destino (donde se guardará el resultado) o el registro fuente secundario (cuyo valor se guardará en memoria, como en el caso de sw) .
- Inmediato (Constante / Offset) [15:0]: 16 bits que representan un valor numérico directo o un desplazamiento (offset)

La distribución de los bits se estructura de la siguiente manera:



Diferencias Principales con las Instrucciones Tipo R

La transición del formato Tipo R al Tipo I implica modificaciones sustanciales en la decodificación lógica :

1. Ausencia del campo funct: Las instrucciones Tipo I carecen de los 6 bits finales de función. El opcode principal es el único responsable de indicarle a la Unidad de Control qué operación matemática debe forzar en la ALU.
2. Destino del Registro: En el formato Tipo R, el resultado siempre se almacena en el registro rd (bits 15:11). En el formato Tipo I, al no existir el campo rd, el registro de destino pasa a ser rt (bits 20:16). Esto obliga al hardware a implementar un multiplexor (RegDst) para decidir qué bits definen la dirección de escritura.
3. Operando Inmediato: Se sacrifica el tercer registro para permitir la incrustación de una constante de 16 bits.

Para el correcto funcionamiento de estas instrucciones, necesitamos agregar un pequeño módulo llamado "Sign extend" o extensor de signo, el cual profundizamos más en un momento.

Extensión de Signo (Sign Extend)

La Unidad Aritmético Lógica (ALU) del procesador MIPS está diseñada para operar exclusivamente con datos de 32 bits de longitud. Dado que el campo inmediato de la instrucción Tipo I aporta únicamente 16 bits, el hardware se enfrenta a una incompatibilidad de tamaño.

Para resolver esto sin alterar el valor matemático (especialmente si el número es negativo en complemento a dos), se implementa un módulo combinacional denominado Extensor de Signo (Sign Extend). Este módulo toma los 16 bits del inmediato y replica el bit más significativo (el bit 15, correspondiente al signo) 16 veces hacia la izquierda para completar una palabra de 32 bits .

En Verilog, esta operación se logra de forma eficiente mediante la concatenación:

```
assign out = {{16{in[15]}}}, in};
```

Instrucciones de Memoria: Load Word (lw) y Store Word (sw)

Las instrucciones lw y sw utilizan el formato Tipo I para implementar el concepto de direccionamiento base más desplazamiento. Lw significa "load word", y en este contexto lo que carga es un dato, de nuestra memoria al banco de registros, su propósito general es obtener variables para hacer operaciones matemáticas o lógicas. Sw significa store word y hace lo contrario a lw, guarda el registro que

tenemos en nuestra memoria de datos, su propósito general es guardar resultados finales o valores temporales en la memoria.

Sintaxis en Ensamblador de Lw y Sw:

lw \$rt, desplazamiento(\$rs) : Carga una palabra (32 bits) de la memoria hacia el registro rt.

sw \$rt, desplazamiento(\$rs) : Guarda la palabra del registro rt en la memoria.

Instrucción Beq:

La instrucción beq (Branch if Equal) en MIPS es de tipo I (Inmediato) y se utiliza para tomar decisiones. Compara el contenido de dos registros; si son iguales, realiza un salto a una etiqueta; si no, continúa con la siguiente instrucción

Funcionamiento:

1. El procesador calcula la dirección de destino de la siguiente forma: Se incrementa el Contador de Programa (PC) para que apunte a la instrucción siguiente
2. El campo "Inmediato" (offset) se extiende de signo a 32 bits y se multiplica por 4 (ya que en MIPS las instrucciones están alineadas a palabras de 4 bytes).
3. Si los registros son iguales, la nueva dirección de ejecución será:
$$PC_nueva = (PC + 4) + (Inmediato \times 4)$$

Si no son iguales, simplemente se ejecuta $PC+4$

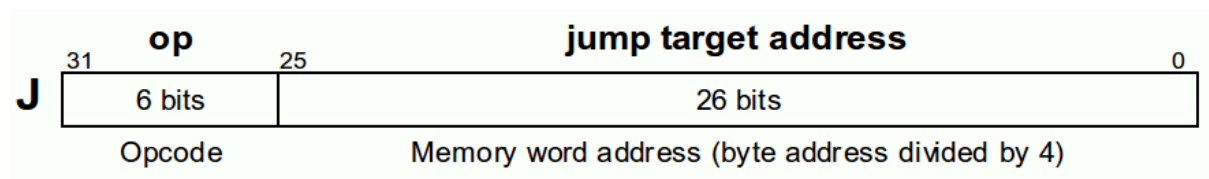
3.7 Instrucciones Tipo J

Las instrucciones Tipo J (Jump) se utilizan en la arquitectura MIPS para realizar saltos incondicionales hacia una dirección específica de la memoria de instrucciones. A diferencia de las instrucciones de salto condicional Tipo I (como beq o bne), que dependen del resultado de una comparación en la ALU para decidir si saltan o no, las instrucciones Tipo J rompen la secuencialidad del programa de forma obligatoria y directa cada vez que son ejecutadas. Su propósito principal es permitir la transferencia del control del flujo del programa hacia subrutinas, funciones o etiquetas lejanas que se encuentran fuera del rango de alcance de los saltos relativos de Tipo I.

Formato de Máquina (Codificación de Bits)

El formato de una instrucción Tipo J consta de 32 bits en total. Para maximizar la cantidad de memoria a la que se puede saltar, este formato prescinde por completo del uso de registros y divide sus campos en únicamente dos apartados:

- Opcode (Código de Operación): Ocupa 6 bits y define la operación específica de salto. Para la instrucción estándar j, el código binario asignado es 000010.
- Target Address (Dirección Objetivo) [25:0]: Ocupa los 26 bits restantes de la instrucción. Contiene la dirección codificada a la que el procesador debe desviar el Program Counter (PC)



Para saltar a una dirección de 32 bits, el procesador reconstruye la posición destino de la siguiente manera:

1. Toma los 26 bits del campo de dirección y los desplaza 2 posiciones a la izquierda (lo que equivale a multiplicar por 4).
2. Toma los 4 bits más significativos del registro Contador de Programa (PC +4).
3. Concatena ambos para formar la dirección final de 32 bits a la que saltará el procesador.

Diferencias Principales con las Instrucciones Tipo R

Es fundamental contrastar el formato Tipo J con el formato Tipo R trabajado en la primera fase para comprender la optimización del hardware:

1. Uso de Registros: Las instrucciones Tipo R operan exclusivamente con datos almacenados en los registros internos (rs, rt, rd). Las instrucciones Tipo J no interactúan con el Banco de Registros; operan directamente sobre el Program Counter (PC) utilizando un valor inmediato .
2. Especificación de la Operación: En el formato Tipo R, el opcode general suele ser 000000, y la operación exacta de la ALU se determina mediante los 6 bits del campo funcional (funct) al final de la instrucción. En el formato Tipo J, el opcode de 6 bits define por sí mismo y de manera absoluta la acción de salto, eliminando la necesidad de un campo funct.

3. Uso de los Bits: Tipo R divide sus 32 bits en 6 campos pequeños para direccionar múltiples registros y desfasamientos (shamt). Tipo J fusiona 26 bits en un solo bloque para disponer del mayor espacio posible para una dirección de memoria.

4. Desarrollo

4.1 Pseudocódigo Algoritmo Ensamblador

Búsqueda Binaria

(Inicialización de variables)

izq <- 0

der <- 0

Mientras NO (der < izq) Hacer:

suma_indices <- izq + der

medio <-

Desplazar_Derecha_Bits (Suma_Indices 1)

Offset_temp <- medio + medio

Offset_final <- Offset_temp + Offset_temp

Direccion_actual <- DireccionBase_A + Offset_final

Si valor_leido == x Entonces:

Retornar medio

Si no, Si valor_leido < x Entonces:

izq <- medio + 1

Si no

der <- medio - 1

Fin Si

Fin Mientras

Retornar -1

4.2 Módulos Modificados (lectura archivo .bin)

Módulo ALU_Control

```
//modificaciones FASE 2
module ALUControl (
    input [5:0] fnc,
    input [2:0] ALUOp,
    output reg [3:0] Sel
);
    always @(*) begin
        case (ALUOp)
            // tipo i
            3'b000: Sel = 4'b0010; // Fuerza SUMA (lw, sw, addi)
            3'b001: Sel = 4'b0110; // Fuerza RESTA (beq, bne, subi)
            3'b011: Sel = 4'b0000; // Fuerza AND (andi)
            3'b100: Sel = 4'b0001; // Fuerza OR (ori)
            3'b101: Sel = 4'b0111; // Fuerza SLT (slti)

            // tipo r
            3'b010: begin
                case (fnc)
                    6'b100000: Sel = 4'b0010; // add
                    6'b100010: Sel = 4'b0110; // sub
                    6'b100100: Sel = 4'b0000; // and
                    6'b100101: Sel = 4'b0001; // or
                    6'b101010: Sel = 4'b0111; // slt
                    6'b100110: Sel = 4'b0011; // xor
                    6'b000000: Sel = 4'b1111; // nop
                    6'b000011: Sel = 4'b1000; // sra
                    6'b000010: Sel = 4'b1001; // srl
                    6'b110100: Sel = 4'b1100; // teq
                    6'b110000: Sel = 4'b1101; // tge
                    default: Sel = 4'b0000;
                endcase
            end

            default: Sel = 4'b0000;
        endcase
    end
endmodule
```

```

1 //modificaciones FASE 2
2 module ALUControl (
3     input [5:0] fnc,
4     input [2:0] ALUOp,
5     output reg [3:0] Sel
6 );
7     always @(*) begin
8         case (ALUOp)
9             // tipo i
10            3'b000: Sel = 4'b0010; // Fuerza SUMA (lw, sw, addi)
11            3'b001: Sel = 4'b0110; // Fuerza RESTA (beq, bne, subi)
12            3'b011: Sel = 4'b0000; // Fuerza AND (andi)
13            3'b100: Sel = 4'b0001; // Fuerza OR (ori)
14            3'b101: Sel = 4'b0111; // Fuerza SLT (slti)
15
16            // tipo r
17            3'b010: begin
18                case (fnc)
19                    6'b100000: Sel = 4'b0010; // add
20                    6'b100010: Sel = 4'b0110; // sub
21                    6'b100100: Sel = 4'b0000; // and
22                    6'b100101: Sel = 4'b0001; // or
23                    6'b101010: Sel = 4'b0111; // slt
24                    6'b100110: Sel = 4'b0011; // xor
25                    6'b000000: Sel = 4'b1111; // nop
26                    6'b000011: Sel = 4'b1000; // sra
27                    6'b000010: Sel = 4'b1001; // srl
28                    6'b110100: Sel = 4'b1100; // teq
29                    6'b110000: Sel = 4'b1101; // tge
30                    default: Sel = 4'b0000;
31                endcase
32            end
33
34            default: Sel = 4'b0000;
35        endcase
36    end
37 endmodule

```

Módulo ALU_Control en ModelSim.

Módulo ALU

```

module ALU (
    input [31:0] A, B,
    input [4:0] shamt, //para sra y srl
    input [3:0] ALUCtrl,
    output reg [31:0] Result,
    output reg Exception //senal de teq y tge
);
    always @(*) begin
        Result = 0;
        Exception = 0;
        case (ALUCtrl)
            4'b0010: Result = A + B;      // add
            4'b0110: Result = A - B;      // sub
            4'b0000: Result = A & B;      // and
            4'b0001: Result = A | B;      //or
            4'b0111: Result = (A < B) ? 1:0; // slt

```

```

4'b0011: Result = A ^ B;      // xor
4'b1000: Result = $signed(B) >>> shamt; // sra
4'b1001: Result = B >> shamt; // srl
4'b1100: Exception = (A == B); // teq
4'b1101: Exception = (A >= B); // tge
4'b1111: Result = 0;          // nop
endcase
end
endmodule

```



```

1  module ALU (
2      input [31:0] A, B,
3      input [4:0] shamt, //para sra y srl
4      input [3:0] ALUCtrl,
5      output reg [31:0] Result,
6      output reg Exception //senal de teq y tge
7  );
8      always @(*) begin
9          Result = 0;
10         Exception = 0;
11         case (ALUCtrl)
12             4'b0010: Result = A + B;          // add
13             4'b0110: Result = A - B;          // sub
14             4'b0000: Result = A & B;          // and
15             4'b0001: Result = A | B;          // or
16             4'b0111: Result = (A < B) ? 1:0; // slt
17             4'b0011: Result = A ^ B;          // xor
18             4'b1000: Result = $signed(B) >>> shamt; // sra
19             4'b1001: Result = B >> shamt;     // srl
20             4'b1100: Exception = (A == B);    // teq
21             4'b1101: Exception = (A >= B);    // tge
22             4'b1111: Result = 0;              // nop
23         endcase
24     end
25 endmodule
26

```

Módulo ALU en ModelSim.

Módulo Banco de Registros

```

module BancoReg (
    input clk,
    input RegWrite,
    input [4:0] rs, rt, rd,
    input [31:0] writeData,
    output [31:0] readData1, readData2
);

reg [31:0] regs [0:31];

//inicializar para evitar el error

```

```

integer i;
initial begin
    for (i = 0; i < 32; i = i + 1) begin
        regs[i] = 32'b0;
    end
end

    assign readData1 = regs[rs];
    assign readData2 = regs[rt];

    always @(posedge clk) begin
        if (RegWrite && rd != 0)
            regs[rd] <= writeData;
    end

endmodule

```



```

1  module BancoReg (
2      input clk,
3      input RegWrite,
4      input [4:0] rs, rt, rd,
5      input [31:0] writeData,
6      output [31:0] readData1, readData2
7  );
8
9      reg [31:0] regs [0:31];
10
11     //inicializar para evitar el error
12     integer i;
13     initial begin
14         for (i = 0; i < 32; i = i + 1) begin
15             regs[i] = 32'b0;
16         end
17     end
18
19     assign readData1 = regs[rs];
20     assign readData2 = regs[rt];
21
22     always @(posedge clk) begin
23         if (RegWrite && rd != 0)
24             regs[rd] <= writeData;
25     end
26
27 endmodule
28

```

Módulo BancoReg en ModelSim.

BUFFERS

Módulo de bufferInstr

```
module BufferInstr(  
    input clk,  
    input reset,  
  
    input [31:0] instr_in,  
    input [31:0] pc4_in,  
  
    output reg [31:0] instr_out,  
    output reg [31:0] pc4_out  
);  
  
always @(posedge clk or posedge reset)  
begin  
    if(reset)  
        begin  
            instr_out <= 32'b0;  
            pc4_out <= 32'b0;  
        end  
    else  
        begin  
            instr_out <= instr_in;  
            pc4_out <= pc4_in;  
        end  
    end  
  
endmodule
```



```

1  module BufferInstr(
2      input clk,
3      input reset,
4
5      input [31:0] instr_in,
6      input [31:0] pc4_in,
7
8      output reg [31:0] instr_out,
9      output reg [31:0] pc4_out
10 );
11
12 always @(posedge clk or posedge reset)
13 begin
14     if(reset)
15     begin
16         instr_out <= 32'b0;
17         pc4_out  <= 32'b0;
18     end
19     else
20     begin
21         instr_out <= instr_in;
22         pc4_out  <= pc4_in;
23     end
24 end
25
26 endmodule

```

Módulo BufferInstr en ModelSim.

Módulo buffer de datos

```

module BufferDatos(
    input clk,
    input reset,

    input [31:0] rd1_in,
    input [31:0] rd2_in,
    input [31:0] imm_in,

    input [4:0] rs_in,
    input [4:0] rt_in,
    input [4:0] rd_in,

    input RegWrite_in,
    input MemToReg_in,
    input MemWrite_in,
    input MemRead_in,
    input ALUSrc_in,
    input RegDst_in,
    input Branch_in,

```

```

    input [2:0] ALUOp_in,

    output reg [31:0] rd1_out,
    output reg [31:0] rd2_out,
    output reg [31:0] imm_out,

    output reg [4:0] rs_out,
    output reg [4:0] rt_out,
    output reg [4:0] rd_out,

    output reg RegWrite_out,
    output reg MemToReg_out,
    output reg MemWrite_out,
    output reg MemRead_out,
    output reg ALUSrc_out,
    output reg RegDst_out,
    output reg Branch_out,

    output reg [2:0] ALUOp_out
);

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        rd1_out <= 0;
        rd2_out <= 0;
        imm_out <= 0;

        rs_out <= 0;
        rt_out <= 0;
        rd_out <= 0;

        RegWrite_out <= 0;
        MemToReg_out <= 0;
        MemWrite_out <= 0;
        MemRead_out <= 0;
        ALUSrc_out <= 0;
        RegDst_out <= 0;
        Branch_out <= 0;

        ALUOp_out <= 0;
    end
    else
    begin
        rd1_out <= rd1_in;
        rd2_out <= rd2_in;

```

```
imm_out <= imm_in;

rs_out <= rs_in;
rt_out <= rt_in;
rd_out <= rd_in;

RegWrite_out <= RegWrite_in;
MemToReg_out <= MemToReg_in;
MemWrite_out <= MemWrite_in;
MemRead_out <= MemRead_in;
ALUSrc_out <= ALUSrc_in;
RegDst_out <= RegDst_in;
Branch_out <= Branch_in;

ALUOp_out <= ALUOp_in;
end
end

endmodule
```

```
module ButterDatos(  
    input clk,  
    input reset,  
  
    input [31:0] rd1_in,  
    input [31:0] rd2_in,  
    input [31:0] imm_in,  
  
    input [4:0] rs_in,  
    input [4:0] rt_in,  
    input [4:0] rd_in,  
  
    input RegWrite_in,  
    input MemToReg_in,  
    input MemWrite_in,  
    input MemRead_in,  
    input ALUSrc_in,  
    input RegDst_in,  
    input Branch_in,  
  
    input [2:0] ALUOp_in,  
  
    output reg [31:0] rd1_out,  
    output reg [31:0] rd2_out,  
    output reg [31:0] imm_out,  
  
    output reg [4:0] rs_out,  
    output reg [4:0] rt_out,  
    output reg [4:0] rd_out,  
  
    output reg RegWrite_out,  
    output reg MemToReg_out,  
    output reg MemWrite_out,  
    output reg MemRead_out,  
    output reg ALUSrc_out,  
    output reg RegDst_out,  
    output reg Branch_out,
```

```

        output reg [2:0] ALUOp_out
    );

    always @(posedge clk or posedge reset)
    begin
        if(reset)
        begin
            rd1_out <= 0;
            rd2_out <= 0;
            imm_out <= 0;

            rs_out <= 0;
            rt_out <= 0;
            rd_out <= 0;

            RegWrite_out <= 0;
            MemToReg_out <= 0;
            MemWrite_out <= 0;
            MemRead_out <= 0;
            ALUSrc_out <= 0;
            RegDst_out <= 0;
            Branch_out <= 0;

            ALUOp_out <= 0;
        end
        else
        begin
            rd1_out <= rd1_in;
            rd2_out <= rd2_in;
            imm_out <= imm_in;

            rs_out <= rs_in;
            rt_out <= rt_in;
            rd_out <= rd_in;

            RegWrite_out <= RegWrite_in;
            MemToReg_out <= MemToReg_in;
            MemWrite_out <= MemWrite_in;
            MemRead_out <= MemRead_in;
            ALUSrc_out <= ALUSrc_in;
            RegDst_out <= RegDst_in;
            Branch_out <= Branch_in;

            ALUOp_out <= ALUOp_in;
        end
    end
endmodule

```

Módulo BufferDatos en ModelSim.

Módulo Buffer Alu

```
module BufferALU(  
    input clk,  
    input reset,  
  
    input [31:0] aluResult_in,  
    input [31:0] writeData_in,  
  
    input [4:0] writeReg_in,  
  
    input Zero_in,  
    input Branch_in,  
  
    input RegWrite_in,  
    input MemToReg_in,  
    input MemWrite_in,  
    input MemRead_in,  
  
    output reg [31:0] aluResult_out,  
    output reg [31:0] writeData_out,  
  
    output reg [4:0] writeReg_out,  
  
    output reg Zero_out,  
    output reg Branch_out,  
  
    output reg RegWrite_out,  
    output reg MemToReg_out,  
    output reg MemWrite_out,  
    output reg MemRead_out  
);  
  
always @(posedge clk or posedge reset)  
begin  
    if(reset)  
    begin  
        aluResult_out <= 0;  
        writeData_out <= 0;  
        writeReg_out <= 0;  
  
        Zero_out <= 0;  
        Branch_out <= 0;  
  
        RegWrite_out <= 0;  
        MemToReg_out <= 0;  
        MemWrite_out <= 0;  
        MemRead_out <= 0;
```

```

end
else
begin
    aluResult_out <= aluResult_in;
    writeData_out <= writeData_in;
    writeReg_out <= writeReg_in;

    Zero_out <= Zero_in;
    Branch_out <= Branch_in;

    RegWrite_out <= RegWrite_in;
    MemToReg_out <= MemToReg_in;
    MemWrite_out <= MemWrite_in;
    MemRead_out <= MemRead_in;
end
end

endmodule

```

PROYECTO FASE 2 > buffers.v

```

118
119 module BufferALU(
120     input clk,
121     input reset,
122
123     input [31:0] aluResult_in,
124     input [31:0] writeData_in,
125
126     input [4:0] writeReg_in,
127
128     input Zero_in,
129     input Branch_in,
130
131     input RegWrite_in,
132     input MemToReg_in,
133     input MemWrite_in,
134     input MemRead_in,
135
136     output reg [31:0] aluResult_out,
137     output reg [31:0] writeData_out,
138
139     output reg [4:0] writeReg_out,
140
141     output reg Zero_out,
142     output reg Branch_out,
143
144     output reg RegWrite_out,
145     output reg MemToReg_out,
146     output reg MemWrite_out,
147     output reg MemRead_out
148 )

```

118
 119
 120
 121
 122
 123
 124
 125
 126
 127
 128
 129
 130
 131
 132
 133
 134
 135
 136
 137
 138
 139
 140
 141
 142
 143
 144
 145
 146
 147
 148

```

150 always @(posedge clk or posedge reset)
151 begin
152     if(reset)
153     begin
154         aluResult_out <= 0;
155         writeData_out <= 0;
156         writeReg_out <= 0;
157
158         Zero_out <= 0;
159         Branch_out <= 0;
160
161         RegWrite_out <= 0;
162         MemToReg_out <= 0;
163         MemWrite_out <= 0;
164         MemRead_out <= 0;
165     end
166     else
167     begin
168         aluResult_out <= aluResult_in;
169         writeData_out <= writeData_in;
170         writeReg_out <= writeReg_in;
171
172         Zero_out <= Zero_in;
173         Branch_out <= Branch_in;
174
175         RegWrite_out <= RegWrite_in;
176         MemToReg_out <= MemToReg_in;
177         MemWrite_out <= MemWrite_in;
178         MemRead_out <= MemRead_in;
179     end
180 end
181
182 endmodule

```



Módulo BufferALU en ModelSim.

Módulo Buffer Final

```

module BufferFinal(
    input clk,
    input reset,

    input [31:0] memData_in,
    input [31:0] aluResult_in,

    input [4:0] writeReg_in,

    input RegWrite_in,
    input MemToReg_in,

    output reg [31:0] memData_out,
    output reg [31:0] aluResult_out,

    output reg [4:0] writeReg_out,

    output reg RegWrite_out,
    output reg MemToReg_out
);

```



```

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        memData_out <= 0;
        aluResult_out <= 0;
        writeReg_out <= 0;

        RegWrite_out <= 0;
        MemToReg_out <= 0;
    end
    else
    begin
        memData_out <= memData_in;
        aluResult_out <= aluResult_in;
        writeReg_out <= writeReg_in;

        RegWrite_out <= RegWrite_in;
        MemToReg_out <= MemToReg_in;
    end
end

endmodule

```

```

186 module BufferFinal(
187     input clk,
188     input reset,
189
190     input [31:0] memData_in,
191     input [31:0] aluResult_in,
192
193     input [4:0] writeReg_in,
194
195     input RegWrite_in,
196     input MemToReg_in,
197
198     output reg [31:0] memData_out,
199     output reg [31:0] aluResult_out,
200
201     output reg [4:0] writeReg_out,
202
203     output reg RegWrite_out,
204     output reg MemToReg_out
205 );
206

```



```

206
207 always @(posedge clk or posedge reset)
208 begin
209     if(reset)
210     begin
211         memData_out <= 0;
212         aluResult_out <= 0;
213         writeReg_out <= 0;
214
215         RegWrite_out <= 0;
216         MemToReg_out <= 0;
217     end
218     else
219     begin
220         memData_out <= memData_in;
221         aluResult_out <= aluResult_in;
222         writeReg_out <= writeReg_in;
223
224         RegWrite_out <= RegWrite_in;
225         MemToReg_out <= MemToReg_in;
226     end
227 end
228
229 endmodule
230

```



Módulo BufferFinal en ModelSim.

Módulo Instruction Fetch Instruction decode

```

module IF_ID(
    input clk,
    input reset,

    input [31:0] instr_in,
    input [31:0] pc4_in,

    output reg [31:0] instr_out,
    output reg [31:0] pc4_out
);

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        instr_out <= 32'b0;
        pc4_out <= 32'b0;
    end
    else
    begin
        instr_out <= instr_in;
        pc4_out <= pc4_in;
    end
end

endmodule

```

```

1  module IF_ID(
2      input clk,
3      input reset,
4
5      input [31:0] instr_in,
6      input [31:0] pc4_in,
7
8      output reg [31:0] instr_out,
9      output reg [31:0] pc4_out
10 );
11
12 always @(posedge clk or posedge reset)
13 begin
14     if(reset)
15     begin
16         instr_out <= 32'b0;
17         pc4_out <= 32'b0;
18     end
19     else
20     begin
21         instr_out <= instr_in;
22         pc4_out <= pc4_in;
23     end
24 end
25
26 endmodule

```



Módulo IF_ID en ModelSim.

Módulo Instruction Fetch Instruction Execute

```

module ID_EX(
    input clk,
    input reset,

    input [31:0] rd1_in,
    input [31:0] rd2_in,
    input [31:0] imm_in,

    input [4:0] rs_in,
    input [4:0] rt_in,
    input [4:0] rd_in,

    input RegWrite_in,
    input MemToReg_in,
    input MemWrite_in,
    input MemRead_in,
    input ALUSrc_in,
    input RegDst_in,
    input Branch_in,

    input [2:0] ALUOp_in,

```

```

output reg [31:0] rd1_out,
output reg [31:0] rd2_out,
output reg [31:0] imm_out,

output reg [4:0] rs_out,
output reg [4:0] rt_out,
output reg [4:0] rd_out,

output reg RegWrite_out,
output reg MemToReg_out,
output reg MemWrite_out,
output reg MemRead_out,
output reg ALUSrc_out,
output reg RegDst_out,
output reg Branch_out,

output reg [2:0] ALUOp_out
);

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        rd1_out <= 0;
        rd2_out <= 0;
        imm_out <= 0;

        rs_out <= 0;
        rt_out <= 0;
        rd_out <= 0;

        RegWrite_out <= 0;
        MemToReg_out <= 0;
        MemWrite_out <= 0;
        MemRead_out <= 0;
        ALUSrc_out <= 0;
        RegDst_out <= 0;
        Branch_out <= 0;

        ALUOp_out <= 0;
    end
    else
    begin
        rd1_out <= rd1_in;
        rd2_out <= rd2_in;
        imm_out <= imm_in;

        rs_out <= rs_in;

```

```

    rt_out <= rt_in;
    rd_out <= rd_in;

    RegWrite_out <= RegWrite_in;
    MemToReg_out <= MemToReg_in;
    MemWrite_out <= MemWrite_in;
    MemRead_out <= MemRead_in;
    ALUSrc_out <= ALUSrc_in;
    RegDst_out <= RegDst_in;
    Branch_out <= Branch_in;

    ALUOp_out <= ALUOp_in;
end
end

endmodule

```

```

28 |
29 | module ID_EX(
30 |     input clk,
31 |     input reset,
32 |
33 |     input [31:0] rd1_in,
34 |     input [31:0] rd2_in,
35 |     input [31:0] imm_in,
36 |
37 |     input [4:0] rs_in,
38 |     input [4:0] rt_in,
39 |     input [4:0] rd_in,
40 |
41 |     input RegWrite_in,
42 |     input MemToReg_in,
43 |     input MemWrite_in,
44 |     input MemRead_in,
45 |     input ALUSrc_in,
46 |     input RegDst_in,
47 |     input Branch_in,
48 |
49 |     input [2:0] ALUOp_in,
50 |
51 |     output reg [31:0] rd1_out,
52 |     output reg [31:0] rd2_out,
53 |     output reg [31:0] imm_out,
54 |
55 |     output reg [4:0] rs_out,
56 |     output reg [4:0] rt_out,
57 |     output reg [4:0] rd_out,
58 |
59 |     output reg RegWrite_out,
60 |     output reg MemToReg_out,
61 |     output reg MemWrite_out,
62 |     output reg MemRead_out,
63 |     output reg ALUSrc_out,
64 |     output reg RegDst_out,
65 |     output reg Branch_out,
66 |

```



```

65     output reg Branch_out,
66
67     output reg [2:0] ALUOp_out
68 );
69
70 always @(posedge clk or posedge reset)
71 begin
72     if(reset)
73     begin
74         rd1_out <= 0;
75         rd2_out <= 0;
76         imm_out <= 0;
77
78         rs_out <= 0;
79         rt_out <= 0;
80         rd_out <= 0;
81
82         RegWrite_out <= 0;
83         MemToReg_out <= 0;
84         MemWrite_out <= 0;
85         MemRead_out <= 0;
86         ALUSrc_out <= 0;
87         RegDst_out <= 0;
88         Branch_out <= 0;
89
90         ALUOp_out <= 0;
91     end
92     else
93     begin
94         rd1_out <= rd1_in;
95         rd2_out <= rd2_in;
96         imm_out <= imm_in;
97
98         rs_out <= rs_in;
99         rt_out <= rt_in;
100        rd_out <= rd_in;
101
102        RegWrite_out <= RegWrite_in;
103        MemToReg_out <= MemToReg_in;
104        MemWrite_out <= MemWrite_in;
105        MemRead_out <= MemRead_in;
106        ALUSrc_out <= ALUSrc_in;
107        RegDst_out <= RegDst_in;
108        Branch_out <= Branch_in;
109
110        ALUOp_out <= ALUOp_in;
111    end
112 end
113
114 endmodule
115

```



Módulo IF_EX en ModelSim.

Módulo Ex Mem

```

module EX_MEM(
    input clk,
    input reset,

    input [31:0] aluResult_in,
    input [31:0] writeData_in,

    input [4:0] writeReg_in,

```

```

    input Zero_in,
    input Branch_in,

    input RegWrite_in,
    input MemToReg_in,
    input MemWrite_in,
    input MemRead_in,

    output reg [31:0] aluResult_out,
    output reg [31:0] writeData_out,

    output reg [4:0] writeReg_out,

    output reg Zero_out,
    output reg Branch_out,

    output reg RegWrite_out,
    output reg MemToReg_out,
    output reg MemWrite_out,
    output reg MemRead_out
);

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        aluResult_out <= 0;
        writeData_out <= 0;
        writeReg_out <= 0;

        Zero_out <= 0;
        Branch_out <= 0;

        RegWrite_out <= 0;
        MemToReg_out <= 0;
        MemWrite_out <= 0;
        MemRead_out <= 0;
    end
    else
    begin
        aluResult_out <= aluResult_in;
        writeData_out <= writeData_in;
        writeReg_out <= writeReg_in;

        Zero_out <= Zero_in;
        Branch_out <= Branch_in;
    end
end

```

```

    RegWrite_out <= RegWrite_in;
    MemToReg_out <= MemToReg_in;
    MemWrite_out <= MemWrite_in;
    MemRead_out <= MemRead_in;
end
end
endmodule

```

```

117 module EX_MEM(
118     input clk,
119     input reset,
120
121     input [31:0] aluResult_in,
122     input [31:0] writeData_in,
123
124     input [4:0] writeReg_in,
125
126     input Zero_in,
127     input Branch_in,
128
129     input RegWrite_in,
130     input MemToReg_in,
131     input MemWrite_in,
132     input MemRead_in,
133
134     output reg [31:0] aluResult_out,
135     output reg [31:0] writeData_out,
136
137     output reg [4:0] writeReg_out,
138
139     output reg Zero_out,
140     output reg Branch_out,
141
142     output reg RegWrite_out,
143     output reg MemToReg_out,
144     output reg MemWrite_out,
145     output reg MemRead_out
146 );

```

```

always @(posedge clk or posedge reset)
begin
    if(reset)
    begin
        aluResult_out <= 0;
        writeData_out <= 0;
        writeReg_out <= 0;

        Zero_out <= 0;
        Branch_out <= 0;

        RegWrite_out <= 0;
        MemToReg_out <= 0;
        MemWrite_out <= 0;
        MemRead_out <= 0;
    end
    else
    begin
        aluResult_out <= aluResult_in;
        writeData_out <= writeData_in;
        writeReg_out <= writeReg_in;

        Zero_out <= Zero_in;
        Branch_out <= Branch_in;

        RegWrite_out <= RegWrite_in;
        MemToReg_out <= MemToReg_in;
        MemWrite_out <= MemWrite_in;
        MemRead_out <= MemRead_in;
    end
end
endmodule

```



Módulo EX_MEM en ModelSim.

Módulo Mem WB

```
module MEM_WB(  
    input clk,  
    input reset,  
  
    input [31:0] memData_in,  
    input [31:0] aluResult_in,  
  
    input [4:0] writeReg_in,  
  
    input RegWrite_in,  
    input MemToReg_in,  
  
    output reg [31:0] memData_out,  
    output reg [31:0] aluResult_out,  
  
    output reg [4:0] writeReg_out,  
  
    output reg RegWrite_out,  
    output reg MemToReg_out  
);  
  
always @(posedge clk or posedge reset)  
begin  
    if(reset)  
    begin  
        memData_out <= 0;  
        aluResult_out <= 0;  
        writeReg_out <= 0;  
  
        RegWrite_out <= 0;  
        MemToReg_out <= 0;  
    end  
    else  
    begin  
        memData_out <= memData_in;  
        aluResult_out <= aluResult_in;  
        writeReg_out <= writeReg_in;  
  
        RegWrite_out <= RegWrite_in;  
        MemToReg_out <= MemToReg_in;  
    end  
end  
  
endmodule
```


);

```
wire [5:0] op = instruction[31:26];  
wire [4:0] rs = instruction[25:21];  
wire [4:0] rt = instruction[20:16];  
wire [4:0] rd = instruction[15:11];  
wire [4:0] sh = instruction[10:6];  
wire [5:0] funct = instruction[5:0];  
wire [15:0] inmed = instruction[15:0];
```

```
wire RegDst, ALUSrc, Branch; // NUEVAS  
wire memToReg, RegWrite, memToWrite, memToRead;  
wire [2:0] ALUOp;  
wire [3:0] ALUCtrl;
```

```
wire aluException; // funciona como la señal "Zero" para el beq  
wire [31:0] readData1, readData2, aluResult;  
wire [31:0] sign_ext_out;  
wire [31:0] idex_rd1;  
wire [31:0] idex_rd2;  
wire [31:0] idex_imm;  
wire idex_ALUSrc;  
wire [31:0] exmem_aluResult;  
wire [31:0] exmem_writeData;
```

```
wire exmem_RegWrite;  
wire exmem_MemWrite;  
wire exmem_MemToReg;  
wire idex_RegWrite;  
wire idex_MemWrite;  
wire idex_MemToReg;  
wire [31:0] memwb_memData;  
wire [31:0] memwb_aluResult;
```

```
wire memwb_RegWrite;  
wire memwb_MemToReg;  
wire [4:0] write_reg_addr;  
wire [31:0] alu_b_in;  
wire [31:0] mem_data_out;  
wire [31:0] write_data_in;
```

```
U_Control UC(  
    .op(op),  
    .funct(funct),  
    .RegDst(RegDst),  
    .ALUSrc(ALUSrc),  
    .Branch(Branch),
```

```

        .memToReg(memToReg),
        .memToWrite(memToWrite),
        .memToRead(memToRead),
        .AluOP(ALUOp),
        .RegWrite(RegWrite)
    );

    ALUControl ALUC(
        .fnc(funct),
        .ALUOp(ALUOp),
        .Sel(ALUCtrl)
    );

    // MUX RegDst fase2
    // decide si el registro destino es rt (0) o rd (1)
    Mux2_1_5 mux_regdst(
        .in0(rt),
        .in1(rd),
        .sel(RegDst),
        .out(write_reg_addr)
    );

    SignExtend se(
        .in(inmed),
        .out(sign_ext_out)
    ); // Convierte el inmediato de 16 bits a 32 bits

    /*Mux2_1_32 mux_alusrc(
        .in0(readData2),
        .in1(sign_ext_out),
        .sel(ALUSrc),
        .out(alu_b_in)
    ); // Decide si la ALU usa readData2 (0) o el inmediato extendido (1)
    */

    BancoReg BR(
        .clk(clk),
        .RegWrite(memwb_RegWrite),
        .rs(rs),
        .rt(rt),
        .rd(write_reg_addr),
        .writeData(write_data_in),
        .readData1(readData1),
        .readData2(readData2)
    );

    ALU alu(
        .A(idex_rd1),
        .B(alu_b_in),

```

```

        .shamt(sh),
        .ALUCtrl(ALUCtrl),
        .Result(aluResult),
        .Exception(aluException)
    );

    BufferDatos idex(
        .clk(clk),
        .reset(1'b0),

        .rd1_in(readData1),
        .rd2_in(readData2),
        .imm_in(sign_ext_out),

        .rs_in(rs),
        .rt_in(rt),
        .rd_in(rd),

        .RegWrite_in(RegWrite),
        .MemToReg_in(memToReg),
        .MemWrite_in(memToWrite),
        .MemRead_in(memToRead),
        .ALUSrc_in(ALUSrc),
        .RegDst_in(RegDst),
        .Branch_in(Branch),

        .ALUOp_in(ALUOp),

        .rd1_out(idex_rd1),
        .rd2_out(idex_rd2),
        .imm_out(idex_imm),

        .rs_out(),
        .rt_out(),
        .rd_out(),

        .RegWrite_out(idex_RegWrite),
        .MemWrite_out(idex_MemWrite),
        .MemToReg_out(idex_MemToReg),
        .MemRead_out(),
        .ALUSrc_out(idex_ALUSrc),
        .RegDst_out(),
        .Branch_out(),

        .ALUOp_out()
    );

    BufferALU exmem(

```

```

.clk(clk),
.reset(1'b0),

.aluResult_in(aluResult),
.writeData_in(idex_rd2),

.writeReg_in(5'b0),

.Zero_in(1'b0),
.Branch_in(1'b0),

.RegWrite_in(idex_RegWrite),
.MemToReg_in(idex_MemToReg),
.MemWrite_in(idex_MemWrite),
.MemRead_in(1'b0),

.aluResult_out(exmem_aluResult),
.writeData_out(exmem_writeData),

.writeReg_out(),

.Zero_out(),
.Branch_out(),

.RegWrite_out(exmem_RegWrite),
.MemToReg_out(exmem_MemToReg),
.MemWrite_out(exmem_MemWrite),
.MemRead_out()
);
Mux2_1_32 mux_alusrc_pipeline(
.in0(idex_rd2),
.in1(idex_imm),
.sel(idex_ALUSrc),
.out(alu_b_in)
);

Mem_D dmem(
.clk(clk),
.MemWrite(exmem_memToWrite),
.dir(exmem_aluResult),
.DEntrada(exmem_writeData), // dato a guardar (para sw)
.DSalida(mem_data_out) // dato extraído (para lw)
);

BufferFinal memwb(
.clk(clk),
.reset(1'b0),

```

```

.memData_in(mem_data_out),
.aluResult_in(exmem_aluResult),

.writeReg_in(5'b0),

.RegWrite_in(exmem_RegWrite),
.MemToReg_in(exmem_MemToReg),

.memData_out(memwb_memData),
.aluResult_out(memwb_aluResult),

.writeReg_out(),

.RegWrite_out(memwb_RegWrite),
.MemToReg_out(memwb_MemToReg)
);

Mux2_1_32 mux_memtoreg(
.in0(memwb_aluResult),
.in1(memwb_memData),
.sel(memwb_MemToReg),
.out(write_data_in)
); // Decide si se guarda en el Banco de Registros el resultado ALU (0) o la
Memoria (1)

// Salida para monitorear en el testbench

assign result = aluResult;
assign Branch_out = Branch;
assign Zero_out = (aluResult == 32'd0) ? 1'b1 : 1'b0; // Si la resta da 0, levanta la
bandera Zero
assign SignExt_out = sign_ext_out;
endmodule

```

```

1 //modificado Fase 2
2 module DPTR (
3     input clk,
4     input [31:0] instruction,
5     output [31:0] result,
6
7
8     output Branch_out,
9     output Zero_out,
10    output [31:0] SignExt_out
11 );
12
13    wire [5:0] op = instruction[31:26];
14    wire [4:0] rs = instruction[25:21];
15    wire [4:0] rt = instruction[20:16];
16    wire [4:0] rd = instruction[15:11];
17    wire [4:0] sh = instruction[10:6];
18    wire [5:0] funct = instruction[5:0];
19    wire [15:0] inmed = instruction[15:0];
20
21    wire RegDst, ALUSrc, Branch; // NUEVAS
22    wire memToReg, RegWrite, memToWrite, memToRead;
23    wire [2:0] ALUOp;
24    wire [3:0] ALUCtrl;
25
26    wire aluException; // funciona como la señal "Zero" para el beq
27    wire [31:0] readData1, readData2, aluResult;
28    wire [31:0] sign_ext_out;
29    wire [31:0] idex_rd1;
30    wire [31:0] idex_rd2;
31    wire [31:0] idex_imm;
32    wire idex_ALUSrc;
33    wire [31:0] exmem_aluResult;
34    wire [31:0] exmem_writeData;
35
36    wire exmem_RegWrite;
37    wire exmem_MemWrite;
38    wire exmem_MemToReg;
39    wire idex_RegWrite;

```



```

40     wire idex_MemWrite;
41     wire idex_MemToReg;
42     wire [31:0] memwb_memData;
43     wire [31:0] memwb_aluResult;
44
45     wire memwb_RegWrite;
46     wire memwb_MemToReg;
47     wire [4:0] write_reg_addr;
48     wire [31:0] alu_b_in;
49     wire [31:0] mem_data_out;
50     wire [31:0] write_data_in;
51
52
53     U_Control UC(
54         .op(op),
55         .funct(funct),
56         .RegDst(RegDst),
57         .ALUSrc(ALUSrc),
58         .Branch(Branch),
59         .memToReg(memToReg),
60         .memToWrite(memToWrite),
61         .memToRead(memToRead),
62         .AluOP(ALUOp),
63         .RegWrite(RegWrite)
64     );
65
66     ALUControl ALUC(
67         .fnc(funct),
68         .ALUOp(ALUOp),
69         .Sel(ALUCtrl)
70     );
71
72     // MUX RegDst fase2
73     // decide si el registro destino es rt (0) o rd (1)
74     Mux2_1_5 mux_regdst(
75         .in0(rt),
76         .in1(rd),

```



```

77     .sel(RegDst),
78     .out(write_reg_addr)
79 );
80
81 SignExtend se(
82     .in(inmed),
83     .out(sign_ext_out)
84 );    // Convierte el inmediato de 16 bits a 32 bits
85
86 /*Mux2_1_32 mux_alusrc(
87     .in0(readData2),
88     .in1(sign_ext_out),
89     .sel(ALUSrc),
90     .out(alu_b_in)
91 );    // Decide si la ALU usa readData2 (0) o el inmediato extendido (1)
92 */
93 BancoReg BR(
94     .clk(clk),
95     .RegWrite(memwb_RegWrite),
96     .rs(rs),
97     .rt(rt),
98     .rd(write_reg_addr),
99     .writeData(write_data_in),
100    .readData1(readData1),
101    .readData2(readData2)
102 );
103
104 ALU alu(
105     .A(idex_rd1),
106     .B(alu_b_in),
107     .shamt(sh),
108     .ALUctrl(ALUctrl),
109     .Result(aluResult),
110     .Exception(aluException)
111 );

```



```

113     BufferDatos idex(
114         .clk(clk),
115         .reset(1'b0),
116
117         .rd1_in(readData1),
118         .rd2_in(readData2),
119         .imm_in(sign_ext_out),
120
121         .rs_in(rs),
122         .rt_in(rt),
123         .rd_in(rd),
124
125         .RegWrite_in(RegWrite),
126         .MemToReg_in(memToReg),
127         .MemWrite_in(memToWrite),
128         .MemRead_in(memToRead),
129         .ALUSrc_in(ALUSrc),
130         .RegDst_in(RegDst),
131         .Branch_in(Branch),
132
133         .ALUOp_in(ALUOp),
134
135         .rd1_out(idex_rd1),
136         .rd2_out(idex_rd2),
137         .imm_out(idex_imm),
138
139         .rs_out(),
140         .rt_out(),
141         .rd_out(),
142
143         .RegWrite_out(idex_RegWrite),
144         .MemWrite_out(idex_MemWrite),
145         .MemToReg_out(idex_MemToReg),
146         .MemRead_out(),
147         .ALUSrc_out(idex_ALUSrc),
148         .RegDst_out(),
149         .Branch_out(),

```



```

150
151 ✓ .ALUOp_out()
152     );
153
154     BufferALU exmem(
155     .clk(clk),
156     .reset(1'b0),
157
158     .aluResult_in(aluResult),
159     .writeData_in(idex_rd2),
160
161     .writeReg_in(5'b0),
162
163     .Zero_in(1'b0),
164     .Branch_in(1'b0),
165
166     .RegWrite_in(idex_RegWrite),
167     .MemToReg_in(idex_MemToReg),
168     .MemWrite_in(idex_MemWrite),
169     .MemRead_in(1'b0),
170
171     .aluResult_out(exmem_aluResult),
172     .writeData_out(exmem_writeData),
173
174     .writeReg_out(),
175
176     .Zero_out(),
177     .Branch_out(),
178
179     .RegWrite_out(exmem_RegWrite),
180     .MemToReg_out(exmem_MemToReg),
181     .MemWrite_out(exmem_MemWrite),
182     .MemRead_out()
183 ✓ );
184     Mux2_1_32 mux_alusrc_pipeline(
185     .in0(idex_rd2),
186     .in1(idex_imm),

```



```

187     .sel(idex_ALUSrc),
188     .out(alu_b_in)
189 );
190
191 Mem_D dmem(
192     .clk(clk),
193     .MemWrite(exmem_memToWrite),
194     .dir(exmem_aluResult),
195     .DEntrada(exmem_writeData),    // dato a guardar (para sw)
196     .DSalida(mem_data_out)        // dato extraído (para lw)
197 );
198
199 BufferFinal memwb(
200     .clk(clk),
201     .reset(1'b0),
202
203     .memData_in(mem_data_out),
204     .aluResult_in(exmem_aluResult),
205
206     .writeReg_in(5'b0),
207
208     .RegWrite_in(exmem_RegWrite),
209     .MemToReg_in(exmem_MemToReg),
210
211     .memData_out(memwb_memData),
212     .aluResult_out(memwb_aluResult),
213
214     .writeReg_out(),
215
216     .RegWrite_out(memwb_RegWrite),
217     .MemToReg_out(memwb_MemToReg)
218 );
219
220 Mux2_1_32 mux_memtoreg(
221     .in0(memwb_aluResult),
222     .in1(memwb_memData),
223     .sel(memwb_MemToReg),
224     .out(write_data_in)
225 ); // Decide si se guarda en el Banco de Registros el resultado ALU (0) o la Memoria (1)
226
227 // Salida para monitorear en el testbench
228
229 assign result = aluResult;
230 assign Branch_out = Branch;
231 assign Zero_out = (aluResult == 32'd0) ? 1'b1 : 1'b0; // Si la resta da 0, levanta la bandera Zero
232 assign SignExt_out = sign_ext_out;
233 endmodule
234

```

Módulo DPTR en ModelSim.

Módulo Mem D

```

module Mem_D(
    input clk,
    input MemWrite,    //1 para escribir, 0 para leer
    input [31:0] dir,
    input [31:0] DEntrada,    output reg [31:0] DSalida
);

```

```

    reg [31:0] MEM [0:255];

    integer i;

    initial begin
        for(i = 0; i < 256; i = i + 1)
            MEM[i] = 32'b0;
        end

    always @*
    begin
        DSalida = MEM[dir];
    end

    always @(posedge clk)
    begin
        if (MemWrite) begin
            MEM[dir] <= DEntrada;
        end
    end

endmodule

```

PROYECTO FASE 2 > Mem_D.v

```

1  module Mem_D(
2      input clk,
3      input MemWrite,      //1 para escribir, 0 para leer
4      input [31:0] dir,
5      input [31:0] DEntrada,    output reg [31:0] DSalida
6  );
7
8      reg [31:0] MEM [0:255];
9
10     integer i;
11
12     initial begin
13         for(i = 0; i < 256; i = i + 1)
14             MEM[i] = 32'b0;
15         end
16
17     always @*
18     begin
19         DSalida = MEM[dir];
20     end
21
22     always @(posedge clk)
23     begin
24         if (MemWrite) begin
25             MEM[dir] <= DEntrada;
26         end
27     end
28
29 endmodule
30
31

```

Módulo MEM_D en ModelSim.

Módulo Mem I

```
module Mem_I(  
    input [31:0] dir,  
    output reg [31:0] DSalida  
);  
  
reg [7:0]MEM[0:255];  
  
initial  
    begin  
        $readmemb("Binary_Code.txt", MEM);  
    end  
  
always @*  
    begin  
        DSalida = {MEM[dir],MEM[dir+1],MEM[dir+2],MEM[dir+3]};  
    end  
endmodule
```



```
PROYECTO FASE 2 > Mem_I.v  
1 module Mem_I(  
2     input [31:0] dir,  
3     output reg [31:0] DSalida  
4 );  
5  
6 reg [7:0]MEM[0:255];  
7  
8 initial  
9     begin  
10         $readmemb("Binary_Code.txt", MEM);  
11     end  
12  
13 always @*  
14     begin  
15         DSalida = {MEM[dir],MEM[dir+1],MEM[dir+2],MEM[dir+3]};  
16     end  
17 endmodule  
18  
19 |
```

Módulo MEM_I en ModelSim.

Módulo MIPS Top Module

```
module MIPS_TM (  
    input clk,  
    input reset,  
    output [31:0] out_result  
);  
    // Cables del PC
```

```

wire [31:0] pc_actual;
wire [31:0] pc_siguiente;
wire [31:0] pc_mas_4;
wire [31:0] pc_branch;

// Cables de instrucciones y control
wire [31:0] instr_to_dp;
wire [31:0] ifid_instr;
wire [31:0] ifid_pc4;
wire branch_sig, zero_sig, pcsource;
wire [31:0] sign_ext_val;

PC pc1(
    .clk(clk),
    .reset(reset),
    .next_pc(pc_siguiente), // Recibe la decisión del Multiplexor
    .pc_out(pc_actual)
);

Mem_I memi(
    .dir(pc_actual),
    .DSalida(instr_to_dp)
);

BufferInstr ifid(
    .clk(clk),
    .reset(reset),

    .instr_in(instr_to_dp),
    .pc4_in(pc_mas_4),

    .instr_out(ifid_instr),
    .pc4_out(ifid_pc4)
);

DPTR dp(
    .clk(clk),
    .instruction(ifid_instr),
    .result(out_result),
    .Branch_out(branch_sig),
    .Zero_out(zero_sig),
    .SignExt_out(sign_ext_val)
);

// Sumador 1: PC + 4 (Avanza a la siguiente instrucción normal)
assign pc_mas_4 = pc_actual + 32'd4;

// Sumador 2: Destino del Branch

```



```
// MIPS desplaza el inmediato 2 bits a la izquierda (equivale a multiplicar por 4)  
assign pc_branch = pc_mas_4 + (sign_ext_val << 2);
```

```
// Compuerta AND: ¿Tenemos instrucción BEQ y los registros son iguales?  
assign pcsource = branch_sig & zero_sig;
```

```
// Multiplexor del PC: Si pcsource es 1, salta; si es 0, avanza normal.  
assign pc_siguiente = pcsource ? pc_branch : pc_mas_4;
```

```
endmodule
```

```

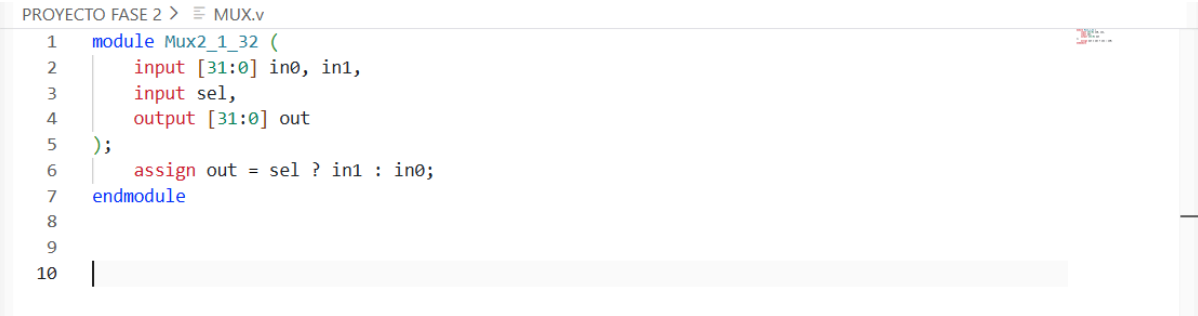
1
2 module MIPS_TM (
3     input clk,
4     input reset,
5     output [31:0] out_result
6 );
7     // Cables del PC
8     wire [31:0] pc_actual;
9     wire [31:0] pc_siguiete;
10    wire [31:0] pc_mas_4;
11    wire [31:0] pc_branch;
12
13    // Cables de instrucciones y control
14    wire [31:0] instr_to_dp;
15    wire [31:0] ifid_instr;
16    wire [31:0] ifid_pc4;
17    wire branch_sig, zero_sig, pcsource;
18    wire [31:0] sign_ext_val;
19
20    PC pc1(
21        .clk(clk),
22        .reset(reset),
23        .next_pc(pc_siguiete), // Recibe la decisión del Multiplexor
24        .pc_out(pc_actual)
25    );
26
27    Mem_I memi(
28        .dir(pc_actual),
29        .DSalida(instr_to_dp)
30    );
31
32    BufferInstr ifid(
33        .clk(clk),
34        .reset(reset),
35
36        .instr_in(instr_to_dp),
37        .pc4_in(pc_mas_4),
38
39        .instr_out(ifid_instr),
40
41        .pc4_out(ifid_pc4)
42    );
43
44    DPTR dp(
45        .clk(clk),
46        .instruction(ifid_instr),
47        .result(out_result),
48        .Branch_out(branch_sig),
49        .Zero_out(zero_sig),
50        .SignExt_out(sign_ext_val)
51    );
52
53    // Sumador 1: PC + 4 (Avanza a la siguiente instrucción normal)
54    assign pc_mas_4 = pc_actual + 32'd4;
55
56    // Sumador 2: Destino del Branch
57    // MIPS desplaza el inmediato 2 bits a la izquierda (equivalente a multiplicar por 4)
58    assign pc_branch = pc_mas_4 + (sign_ext_val << 2);
59
60    // Compuerta AND: ¿Tenemos instrucción BEQ y los registros son iguales?
61    assign pcsource = branch_sig & zero_sig;
62
63    // Multiplexor del PC: Si pcsource es 1, salta; si es 0, avanza normal.
64    assign pc_siguiete = pcsource ? pc_branch : pc_mas_4;
65
66 endmodule

```

Módulo MIPS_TM en ModelSim.

Multiplexor 2 a 1 32 bits

```
module Mux2_1_32 (  
    input [31:0] in0, in1,  
    input sel,  
    output [31:0] out  
);  
    assign out = sel ? in1 : in0;  
endmodule
```

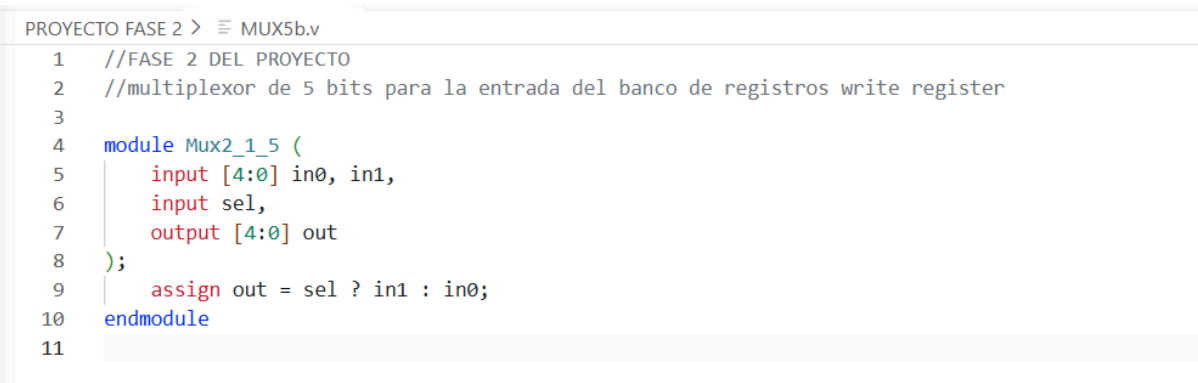
A screenshot of the ModelSim IDE showing a Verilog file named MUX.v. The code defines a module Mux2_1_32 with two 32-bit inputs (in0, in1), a 1-bit select input (sel), and a 32-bit output (out). The output is assigned as out = sel ? in1 : in0. The code is displayed with line numbers 1 through 10 on the left margin.

```
PROYECTO FASE 2 > MUX.v  
1 module Mux2_1_32 (  
2     input [31:0] in0, in1,  
3     input sel,  
4     output [31:0] out  
5 );  
6     assign out = sel ? in1 : in0;  
7 endmodule  
8  
9  
10
```

Módulo Mux2_1_32 en ModelSim.

Multiplexor 2 a 1 5 bits

```
//FASE 2 DEL PROYECTO  
//multiplexor de 5 bits para la entrada del banco de registros write register  
  
module Mux2_1_5 (  
    input [4:0] in0, in1,  
    input sel,  
    output [4:0] out  
);  
    assign out = sel ? in1 : in0;  
endmodule
```

A screenshot of the ModelSim IDE showing a Verilog file named MUX5b.v. The code includes comments indicating it's for Phase 2 of the project and is a 5-bit multiplexer for the write register bank input. It defines a module Mux2_1_5 with two 5-bit inputs (in0, in1), a 1-bit select input (sel), and a 5-bit output (out). The output is assigned as out = sel ? in1 : in0. The code is displayed with line numbers 1 through 11 on the left margin.

```
PROYECTO FASE 2 > MUX5b.v  
1 //FASE 2 DEL PROYECTO  
2 //multiplexor de 5 bits para la entrada del banco de registros write register  
3  
4 module Mux2_1_5 (  
5     input [4:0] in0, in1,  
6     input sel,  
7     output [4:0] out  
8 );  
9     assign out = sel ? in1 : in0;  
10 endmodule  
11
```

Módulo Mux2_1_5 en ModelSim.

Módulo Program Counter

```
module PC (  
    input clk,  
    input reset,  
    input [31:0] next_pc,    // MIPS_TM le dira a donde ir  
    output reg [31:0] pc_out  
);  
    always @(posedge clk or posedge reset) begin  
        if (reset)  
            pc_out <= 32'b0;  
        else  
            pc_out <= next_pc;  
        end  
    endmodule
```



```
PROYECTO FASE 2 > Program_Counter.v  
1  
2 module PC (  
3     input clk,  
4     input reset,  
5     input [31:0] next_pc,    // MIPS_TM le dira a donde ir  
6     output reg [31:0] pc_out  
7 );  
8     always @(posedge clk or posedge reset) begin  
9         if (reset)  
10            pc_out <= 32'b0;  
11        else  
12            pc_out <= next_pc;  
13        end  
14    endmodule  
15
```

Módulo PC en ModelSim.

Módulo Sign Extend

//FASE 2 DEL PROYECTO
//este extensor de signo rellena con 0 o con 1 para hacer que la instruccion sea de 32 bits

```
module SignExtend (  
    input [15:0] in,  
    output [31:0] out  
);  
    // Replica el bit de signo (el bit 15) 16 veces hacia la izquierda  
    assign out = {{16{in[15]}}, in};  
endmodule
```

```

P C:\Users\emman\Downloads\PROYECTO FASE 2\PROYECTO FASE 2\buffers.v
1 //FASE 2 DEL PROYECTO
2 //este extensor de signo rellena con 0 o con 1 para hacer que la instruccion sea de 32 bits
3
4 module SignExtend (
5     input [15:0] in,
6     output [31:0] out
7 );
8     // Replica el bit de signo (el bit 15) 16 veces hacia la izquierda
9     assign out = {{16{in[15]}}, in};
10 endmodule
11
12

```

Módulo SignExtend en ModelSim.

Módulo Test Bench DPTR

```

module TB_DPTR;
    reg clk;
    reg reset;
    wire [31:0] result;

    MIPS_TM tm(clk, reset, result);

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        reset = 1;
        #1;

        tm.dp.BR.regs[8] = 32'd20;
        tm.dp.BR.regs[9] = 32'd40;
        tm.dp.BR.regs[10] = 32'd60;
        tm.dp.BR.regs[11] = 32'd80;
        tm.dp.BR.regs[12] = 32'd5;
        tm.dp.BR.regs[13] = 32'd15;
        tm.dp.BR.regs[17] = 32'd100;
        tm.dp.BR.regs[18] = 32'd50;
        tm.dp.BR.regs[19] = 32'd25;
        tm.dp.BR.regs[20] = 32'd30;
        tm.dp.BR.regs[21] = 32'd10;

        tm.dp.BR.regs[0] = 32'd0;

        #9;
        reset = 0;
    end
endmodule

```

```

    $display("Ejecutando Archivo .bin");

    #150;

    $display("Simulacion finalizada");

    $stop;
end
endmodule

```

```

C:/Users/karla/OneDrive/Documentos/ArquiDeCompu/TB_DPTR.v (/TB_DPTR) - Default
Ln#
1  module TB_DPTR;
2      reg clk;
3      reg reset;
4      wire [31:0] result;
5
6      MIPS_TM tm(clk, reset, result);
7
8      initial begin
9          clk = 0;
10         forever #5 clk = ~clk;
11     end
12
13     initial begin
14         reset = 1;
15         #1;
16
17         tm.dp.BR.regs[8]  = 32'd20;
18         tm.dp.BR.regs[9]  = 32'd40;
19         tm.dp.BR.regs[10] = 32'd60;
20         tm.dp.BR.regs[11] = 32'd80;
21         tm.dp.BR.regs[12] = 32'd5;
22         tm.dp.BR.regs[13] = 32'd15;
23         tm.dp.BR.regs[17] = 32'd100;
24         tm.dp.BR.regs[18] = 32'd50;
25         tm.dp.BR.regs[19] = 32'd25;
26         tm.dp.BR.regs[20] = 32'd30;
27         tm.dp.BR.regs[21] = 32'd10;
28
29         tm.dp.BR.regs[0]  = 32'd0;
30
31         #9;
32         reset = 0;
33
34         $display("Ejecutando Archivo .bin");
35
36         #150;
37
38         $display("Simulacion finalizada");
39
40         $stop;
41     end
42 endmodule

```

Módulo TB_DPTR en ModelSim.

Módulo Unidad de Control

```
module U_Control(  
    input [5:0] op,  
    input [5:0] funct,  
    // Señales fase 2  
    output reg RegDst,  
    output reg ALUSrc,  
    output reg Branch,  
    // fase 1  
    output reg memToReg,  
    output reg memToWrite,  
    output reg memToRead,  
    output reg [2:0] AluOP,  
    output reg RegWrite  
);  
  
always@(*) begin  
    // Valores por defecto  
    RegDst = 1'b0; ALUSrc = 1'b0; Branch = 1'b0;  
    RegWrite = 1'b0; AluOP = 3'b000; memToRead = 1'b0;  
    memToWrite = 1'b0; memToReg = 1'b0;  
  
    case (op)  
        // TIPO R  
        6'b000000: begin  
            RegDst = 1'b1; // Destino es rd  
            ALUSrc = 1'b0;  
            AluOP = 3'b010; // Código para ir al case(funct)  
  
            if(funct == 6'b110100 || funct == 6'b110000) // para teq y tge  
                RegWrite = 1'b0;  
            else  
                RegWrite = 1'b1;  
        end  
  
        // LW (Load Word)  
        6'b100011: begin  
            RegDst = 1'b0;  
            ALUSrc = 1'b1;  
            memToReg = 1'b1; // Viene de Memoria  
            RegWrite = 1'b1;  
            memToRead = 1'b1;  
            AluOP = 3'b000; // Suma  
        end  
  
        // SW (Store Word)
```

```

6'b101011: begin
    ALUSrc = 1'b1;
    memToWrite = 1'b1; // Escribe en Memoria
    AluOP = 3'b000; // Suma
end

```

```

// BEQ (Branch if Equal)
6'b000100: begin
    ALUSrc = 1'b0;
    Branch = 1'b1; // Enciende el salto
    AluOP = 3'b001; // Resta para comparar
end

```

```

// ADDI
6'b001000: begin
    RegDst = 1'b0;
    ALUSrc = 1'b1;
    RegWrite = 1'b1;
    AluOP = 3'b000; // ALUControl hará SUMA
end

```

```

// ANDI
6'b001100: begin
    RegDst = 1'b0;
    ALUSrc = 1'b1;
    RegWrite = 1'b1;
    AluOP = 3'b011; // ALUControl hará AND
end

```

```

// ORI
6'b001101: begin
    RegDst = 1'b0;
    ALUSrc = 1'b1;
    RegWrite = 1'b1;
    AluOP = 3'b100; // ALUControl hará OR
end

```

```

// SLTI
6'b001010: begin
    RegDst = 1'b0;
    ALUSrc = 1'b1;
    RegWrite = 1'b1;
    AluOP = 3'b101;
end

```

```

6'b001001: begin
    RegDst = 1'b0;
    ALUSrc = 1'b1;

```



```
    RegWrite = 1'b1;  
    AluOP = 3'b001; // ALUControl hará RESTA  
end
```

```
    endcase  
end  
endmodule
```

```

1
2 module U_Control(
3     input [5:0] op,
4     input [5:0] funct,
5     // Señales fase 2
6     output reg RegDst,
7     output reg ALUSrc,
8     output reg Branch,
9     // fase 1
10    output reg memToReg,
11    output reg memToWrite,
12    output reg memToRead,
13    output reg [2:0] AluOP,
14    output reg RegWrite
15 );|
16
17 always@(*) begin
18     // Valores por defecto
19     RegDst = 1'b0; ALUSrc = 1'b0; Branch = 1'b0;
20     RegWrite = 1'b0; AluOP = 3'b000; memToRead = 1'b0;
21     memToWrite = 1'b0; memToReg = 1'b0;
22
23     case (op)
24         // TIPO R
25         6'b000000: begin
26             RegDst = 1'b1;    // Destino es rd
27             ALUSrc = 1'b0;
28             AluOP = 3'b010;    // Código para ir al case(funct)
29
30             if(funct == 6'b110100 || funct == 6'b110000) // para teq y tge
31                 RegWrite = 1'b0;
32             else
33                 RegWrite = 1'b1;
34         end
35
36         // LW (Load Word)
37         6'b100011: begin
38             RegDst = 1'b0;
39             ALUSrc = 1'b1;

```

```

40         memToReg = 1'b1; // Viene de Memoria
41         RegWrite = 1'b1;
42         memToRead = 1'b1;
43         AluOP = 3'b000; // Suma
44     end
45
46     // SW (Store Word)
47     6'b101011: begin
48         ALUSrc = 1'b1;
49         memToWrite = 1'b1; // Escribe en Memoria
50         AluOP = 3'b000; // Suma
51     end
52
53     // BEQ (Branch if Equal)
54     6'b000100: begin
55         ALUSrc = 1'b0;
56         Branch = 1'b1; // Enciende el salto
57         AluOP = 3'b001; // Resta para comparar
58     end
59
60     // ADDI
61     6'b001000: begin
62         RegDst = 1'b0;
63         ALUSrc = 1'b1;
64         RegWrite = 1'b1;
65         AluOP = 3'b000; // ALUControl hará SUMA
66     end
67
68     // ANDI
69     6'b001100: begin
70         RegDst = 1'b0;
71         ALUSrc = 1'b1;
72         RegWrite = 1'b1;
73         AluOP = 3'b011; // ALUControl hará AND
74     end
75
76     // ORI
77     6'b001101: begin
78         RegDst = 1'b0;
79         ALUSrc = 1'b1;
80         RegWrite = 1'b1;
81         AluOP = 3'b100; // ALUControl hará OR
82     end
83
84     // SLTI
85     6'b001010: begin
86         RegDst = 1'b0;
87         ALUSrc = 1'b1;
88         RegWrite = 1'b1;
89         AluOP = 3'b101;
90     end
91
92     6'b001001: begin
93         RegDst = 1'b0;
94         ALUSrc = 1'b1;
95         RegWrite = 1'b1;
96         AluOP = 3'b001; // ALUControl hará RESTA
97     end
98
99     endcase
100 end
101 endmodule

```

Módulo U_Control en ModelSim.

4.3 Ensamblador, script Python y .bin

Ensamblador (.asm)

```
# PRUEBA DE INSTRUCCIONES TIPO R

# Aritméticas y Lógicas (rd, rs, rt)
add $t0, $s1, $s2
sub $t1, $t0, $s3
and $t2, $s4, $s5
or  $t3, $t1, $t2
xor $t4, $t2, $t3
slt $t5, $s1, $s2

# Desplazamientos (rd, rt, shamt) — rs queda en 00000
srl $s6, $t4, 2
sra $s7, $t5, 4

# Trampas (rs, rt) — rd y shamt quedan en 00000
teq $s1, $s2
tge $t0, $t1

# Operación Nula
nop

# PRUEBA DE INSTRUCCIONES TIPO I

# Aritméticas / lógicas con inmediato (rt, rs, imm)
addi $t0, $zero, 10  # reg 8 -> decimal debe ser 10
slti $t1, $zero, 10  # reg 9 -> decimal debe ser 1
andi $t2, $zero, 10  # reg 10 -> decimal debe ser 0
ori  $t3, $zero, 10  # reg 11 -> decimal debe ser 1

# Memoria — sw/lw usan formato: reg_dato, offset(reg_base)
sw $t0, 0($zero)     # guardar reg 8 (10) en mem[0]
lw $t4, 0($zero)     # cargar  mem[0] (10) en reg 12

# Salto condicional (rs, rt, etiqueta o offset)
beq $zero, $zero, -1  # salta al inicio (offset -1 => 0xFFFF)
```

Desglose de Instrucciones en Ensamblador

instrucciones Tipo R (Aritméticas, Lógicas y Especiales)

Las instrucciones de este bloque emplean el formato de tres registros, donde las direcciones de los operandos se extraen de los campos **rs (bits 25:21)** y **rt (bits 20:16)**, y el almacenamiento final se efectúa en el campo **rd (bits 15:11)**.

- **add \$t0, \$s1, \$s2** (Suma Aritmética):
La ALU recibe los valores contenidos en los registros fuente \$s1 y \$s2 a través de los canales de lectura del Banco de Registros. Ejecuta una adición aritmética y el resultado se almacena en el registro de destino temporal \$t0 (registro 8).
- **sub \$t1, \$t0, \$s3** (Resta Aritmética):
La ALU sustrae el valor almacenado en el registro fuente \$s3 al contenido previo del registro temporal \$t0. El resultado de la diferencia se deposita en el registro de destino \$t1 (registro 9).
- **and \$t2, \$s4, \$s5** (Operación Lógica AND):
Se realiza una comparación lógica bit a bit empleando la compuerta AND entre los registros fuente \$s4 y \$s5. El patrón de bits resultante se almacena en el registro \$t2 (registro 10).
- **or \$t3, \$t1, \$t2** (Operación Lógica OR):
El procesador evalúa bit a bit los contenidos de los registros temporales \$t1 y \$t2 mediante la operación lógica OR. El resultado se guarda en el registro de destino \$t3 (registro 11).
- **xor \$t4, \$t2, \$t3** (Operación Lógica XOR):
Se ejecuta una operación OR exclusiva bit a bit entre los operandos de los registros \$t2 y \$t3. El resultado final se escribe en el registro \$t4 (registro 12).
- **slt \$t5, \$s1, \$s2** (Comparación Menor Que):
La instrucción Set on Less Than evalúa si el valor numérico en \$s1 es estrictamente menor que el de \$s2. Al cumplirse la condición, la ALU devuelve un 1 lógico; de lo contrario, devuelve un 0. Este valor booleano se almacena en el registro \$t5 (registro 13).
- **srl \$s6, \$t4, 2** (Desplazamiento Lógico a la Derecha):
Toma los bits del registro fuente \$t4 (asignado al campo rt) y los desplaza dos posiciones hacia la derecha, rellenando los espacios vacíos del lado izquierdo con ceros lógicos. El campo rs se mantiene en 00000 y el valor

modificado se guarda en el registro de destino \$s6 (registro 22) mediante el valor constante dictado por el campo shamt (bits 10:6).

- **sra \$s7, \$t5, 4** (Desplazamiento Aritmético a la Derecha):
Desplaza los bits del registro fuente \$t5 cuatro posiciones hacia la derecha. A diferencia de srl, este módulo preserva el bit de signo original (bit 31) replicándolo en las posiciones de la izquierda, garantizando la integridad matemática de las cifras negativas en complemento a dos. El resultado se almacena en \$s7 (registro 23).
- **teq \$s1, \$s2** (Excepción por Igualdad):
Compara los contenidos de los registros \$s1 y \$s2. Si ambos valores son exactamente iguales, la ALU genera una bandera activa en su salida Exception. Al ser una instrucción de trampa, la Unidad de Control mantiene la señal RegWrite = 0, evitando modificar el Banco de Registros (rd y shamt quedan en ceros).
- **tge \$t0, \$t1** (Excepción por Mayor o Igual):
Evalúa si el valor almacenado en \$t0 es mayor o igual al de \$t1. Si la condición matemática se cumple, se dispara la señal de excepción en el sistema, bloqueando la escritura en los registros.
- **nop** (Operación Nula):
Representa la ausencia de operación (No Operation). Físicamente, el procesador la interpreta como un desfase lógico sll \$0, \$0, 0. No altera el estado de las memorias ni de los registros del sistema, consumiendo un ciclo de reloj para mantener el sincronismo temporal.

Instrucciones Tipo I (Inmediatas y Acceso a Memoria)

Las instrucciones de este bloque redistribuyen sus campos para operar un registro fuente rs (bits 25:21) contra una constante numérica de 16 bits (bits 15:0) llamada inmediato, almacenando el resultado en el campo rt (bits 20:16)

- **addi \$t0, \$zero, 10** (Suma Inmediata): El valor constante de 16 bits (10) es procesado por el módulo SignExtend para convertirse en una palabra de 32 bits. La ALU realiza la adición de este valor con el contenido del registro \$zero (cuyo valor físico es de forma absoluta cero). El resultado resultante (10 decimal) se almacena en el registro destino \$t0 (registro 8).
- **slti \$t1, \$zero, 10** (Comparación Inmediata Menor Que): Compara si el valor del registro \$zero (0) es menor que la constante extendida de 32 bits (10). Debido a que la condición es verdadera, la ALU emite un 1 lógico, el cual es direccionado y escrito en el registro destino \$t1 (registro 9).

- **andi \$t2, \$zero, 10** (AND Lógico con Inmediato): Ejecuta una operación AND bit a bit entre la constante extendida (10) y el registro \$zero (constituido por puros ceros). Al operar cualquier combinación contra ceros, el resultado lógico final es 0, almacenándose en el registro \$t2 (registro 10).
- **ori \$t3, \$zero, 10** (OR Lógico con Inmediato): Realiza una operación OR bit a bit entre el registro \$zero (0) y el inmediato extendido (10). Dado que cualquier bit operado con OR contra 0 mantiene su estado original, la ALU genera como salida un 10 decimal. Este valor se escribe en el registro destino \$t3 (registro 11). (Nota de depuración: Se corrige el comentario original del archivo de pruebas que indicaba un valor esperado de 1).
- **sw \$t0, 0(\$zero)** (Almacenamiento de Palabra en Memoria): La ALU calcula la dirección física efectiva sumando el contenido del registro base \$zero (0) más el desplazamiento (offset) inmediato de 0 bits. Con la señal de control MemWrite = 1, el procesador toma la palabra de 32 bits almacenada en el registro \$t0 (que vale 10) y la escribe directamente en la celda correspondiente de la Memoria de Datos (Mem_D[0]).
- **lw \$t4, 0(\$zero)** (Carga de Palabra desde Memoria): Calcula la dirección de lectura sumando el registro base \$zero (0) y el desfaseamiento de 0. Al activarse la señal MemRead = 1, la memoria Mem_D expulsa el dato guardado en esa posición (el número 10) a través del canal DSalida. El multiplexor MemToReg intercepta este valor y lo transfiere al puerto de escritura del Banco de Registros, guardándolo en el registro destino \$t4 (registro 12).
- **beq \$zero, \$zero, -1** (Salto Condicional por Igualdad): La ALU efectúa una resta interna entre los operandos de los registros \$zero y \$zero. Al ser la diferencia igual a cero, la ALU activa su bandera interna Zero_out, informando al módulo superior MIPS_TM. Al estar la señal Branch encendida por la Unidad de Control, la compuerta AND activa el multiplexor del PC, obligando al sistema a romper el flujo secuencial y efectuar un salto relativo calculado mediante la ecuación $PC + 4 + (inmed \ll 2)$ redirigiendo el procesamiento hacia la dirección indicada por el desfaseamiento negativo de la etiqueta.

Script de Python (Conversor)

```
import tkinter as tk
from tkinter import filedialog, messagebox
```

```
# — Tablas MIPS
```

```

REG = {
    "$zero": "00000", "$at": "00001", "$v0": "00010", "$v1": "00011",
    "$a0": "00100", "$a1": "00101", "$a2": "00110", "$a3": "00111",
    "$t0": "01000", "$t1": "01001", "$t2": "01010", "$t3": "01011",
    "$t4": "01100", "$t5": "01101", "$t6": "01110", "$t7": "01111",
    "$s0": "10000", "$s1": "10001", "$s2": "10010", "$s3": "10011",
    "$s4": "10100", "$s5": "10101", "$s6": "10110", "$s7": "10111",
    "$t8": "11000", "$t9": "11001",
}

# Tipo R – código de función (6 bits)
FUNCT = {
    "add": "100000", "sub": "100010", "and": "100100", "or": "100101",
    "xor": "100110", "slt": "101010", "srl": "000010", "sra": "000011",
    "lge": "110000", "teq": "110100",
}

# Tipo I – opcode (6 bits)
OPCODE_I = {
    "addi": "001000",
    "slti": "001010",
    "andi": "001100",
    "ori": "001101",
    "lw": "100011",
    "sw": "101011",
    "beq": "000100",
}

# ——— Helpers

```

```

def to_bin(val, bits):
    """Entero (con signo) a cadena binaria de `bits` dígitos."""
    v = int(val)
    if v < 0:
        # complemento a 2
        v = v & ((1 << bits) - 1)
    return format(v, f'0{bits}b')

def parse_mem(operand):
    """
    Descompone 'offset(base)' → (offset_str, base_reg_str).
    Ejemplos: '0($zero)' → ('0', '$zero')
              '-4($sp)' → ('-4', '$sp')
    """
    operand = operand.strip()
    if '(' in operand:
        offset_str, rest = operand.split('(', 1)
        base_reg = rest.rstrip(')')
        return offset_str.strip(), base_reg.strip()
    raise ValueError(f"Formato de memoria inválido: '{operand}'")

```


— Ensamblador de una línea

```
def ensamblar_linea(linea, num_linea=None, labels=None, linea_actual=None):
```

```
    """
```

```
    Convierte una línea de ensamblador MIPS a una cadena de 32 bits.
```

```
    Retorna None si la línea está vacía/es comentario/es etiqueta.
```

```
    Retorna una cadena 'ERROR ...' en caso de problema.
```

```
    """
```

```
    # Quitar comentarios y normalizar
```

```
    linea = linea.split('#')[0].replace(',', ' ').strip().lower()
```

```
    if not linea or linea.endswith(':'): # vacía o solo etiqueta
```

```
        return None
```

```
    # Si hay etiqueta al inicio de la instrucción, quitarla
```

```
    if ':' in linea:
```

```
        linea = linea.split(':', 1)[1].strip()
```

```
    if not linea:
```

```
        return None
```

```
    partes = linea.split()
```

```
    inst = partes[0]
```

```
    # — NOP
```

```
    if inst == "nop":
```

```
        return "0" * 32
```

```
    opcode = "000000" # Tipo R por defecto
```

```
    try:
```

```
        # — TIPO R: aritméticas/lógicas
```

```
        if inst in ("add", "sub", "and", "or", "xor", "slt"):
```

```
            rd, rs, rt = partes[1], partes[2], partes[3]
```

```
            return opcode + REG[rs] + REG[rt] + REG[rd] + "00000" + FUNCT[inst]
```

```
        # — TIPO R: desplazamientos
```

```
        elif inst in ("srl", "sra"):
```

```
            rd, rt, shamt = partes[1], partes[2], partes[3]
```

```
            return opcode + "00000" + REG[rt] + REG[rd] + to_bin(shamt, 5) + FUNCT[inst]
```

```
        # — TIPO R: trampas
```

```
        elif inst in ("tge", "teq"):
```

```
            rs, rt = partes[1], partes[2]
```

```
            return opcode + REG[rs] + REG[rt] + "00000" + "00000" + FUNCT[inst]
```

```
        # — TIPO I: addi / slti / andi / ori → rt, rs, imm
```

```
        # Sintaxis ensamblador: inst $rt, $rs, inmediato
```

```
        elif inst in ("addi", "slti", "andi", "ori"):
```

```

    rt, rs, imm = partes[1], partes[2], partes[3]
    return OPCODE_I[inst] + REG[rs] + REG[rt] + to_bin(imm, 16)

# — TIPO I: lw → opcode rs rt offset(16 bits) —————
# Sintaxis ensamblador: lw $rt, offset($rs)
elif inst == "lw":
    rt = partes[1]
    offset_str, rs = parse_mem(partes[2])
    return OPCODE_I["lw"] + REG[rs] + REG[rt] + to_bin(offset_str, 16)

# — TIPO I: sw → opcode rs rt offset(16 bits) —————
# Sintaxis ensamblador: sw $rt, offset($rs)
elif inst == "sw":
    rt = partes[1]
    offset_str, rs = parse_mem(partes[2])
    return OPCODE_I["sw"] + REG[rs] + REG[rt] + to_bin(offset_str, 16)

# — TIPO I: beq → opcode rs rt offset(16 bits) —————
# Sintaxis ensamblador: beq $rs, $rt, offset_numerico
# (p.ej. beq $zero, $zero, -1 para apuntar a la instrucción anterior)
elif inst == "beq":
    rs, rt, offset = partes[1], partes[2], partes[3]
    return OPCODE_I["beq"] + REG[rs] + REG[rt] + to_bin(offset, 16)

except KeyError as e:
    return f"ERROR línea {num_linea}: registro desconocido {e} → '{linea}'"
except Exception as e:
    return f"ERROR línea {num_linea}: {e} → '{linea}'"

return f"ERROR línea {num_linea}: instrucción no reconocida → '{inst}'"

```

— GUI

```

def seleccionar_archivo():
    ruta = filedialog.askopenfilename(
        filetypes=[("MIPS Assembly", "*.asm"), ("Text files", "*.txt")]
    )
    if ruta:
        entrada_ruta.delete(0, tk.END)
        entrada_ruta.insert(0, ruta)

def procesar_archivo():
    ruta_origen = entrada_ruta.get()
    if not ruta_origen:
        messagebox.showwarning("Atención", "Primero carga un archivo .asm")
        return
    try:
        with open(ruta_origen, 'r') as f:
            lineas = f.readlines()

```

```

resultado = []
errores = []
for i, l in enumerate(lineas, start=1):
    binario = ensamblar_linea(l, num_linea=i)
    if binario is None:
        continue # línea vacía / comentario / etiqueta
    if binario.startswith("ERROR"):
        errores.append(binario)
    else:
        resultado.append(binario)

if errores:
    detalle = "\n".join(errores)
    if not messagebox.askyesno(
        "Advertencia",
        f"Se encontraron {len(errores)} error(es).\n\n{detalle}\n\n"
        "¿Deseas guardar de todas formas las instrucciones válidas?"
    ):
        return

ruta_destino = filedialog.asksaveasfilename(
    defaultextension=".txt",
    filetypes=[("Text file", "*.txt"), ("Binary file", "*.bin")]
)
if ruta_destino:
    with open(ruta_destino, 'w') as f:
        lineas_8 = []
        for instruccion in resultado:
            for i in range(0, 32, 8):
                lineas_8.append(instruccion[i:i+8])
            f.write("\n".join(lineas_8))
    messagebox.showinfo(
        "Éxito",
        f"Archivo generado con {len(resultado)} instrucción(es).\n\n{ruta_destino}"
    )

except Exception as e:
    messagebox.showerror("Error", f"No se pudo procesar: {e}")

```

— Ventana principal

```

root = tk.Tk()
root.title("MIPS Ensamblador – Tipo R + Tipo I")
root.geometry("540x260")
root.resizable(False, False)

tk.Label(root, text="Conversor ASM → Binario (R + I)",
        font=("Arial", 14, "bold")).pack(pady=12)

frame_carga = tk.Frame(root)
frame_carga.pack(pady=8)

```

```

btn_cargar = tk.Button(frame_carga, text="Cargar .asm",
                        command=seleccionar_archivo, width=12)
btn_cargar.pack(side=tk.LEFT, padx=6)

entrada_ruta = tk.Entry(frame_carga, width=42)
entrada_ruta.pack(side=tk.LEFT, padx=6)

btn_convertir = tk.Button(
    root, text="CONVERTIR A BINARIO",
    bg="#4CAF50", fg="white", font=("Arial", 10, "bold"),
    command=procesar_archivo
)
btn_convertir.pack(pady=18, ipadx=12, ipady=6)

tk.Label(root,
        text="Instrucciones soportadas – Tipo R: add sub and or xor slt srl sra teq tge nop\n"
        "Tipo I: addi slti andi ori lw sw beq",
        font=("Arial", 8), fg="gray").pack()

root.mainloop()

```

Archivo en Binario (.bin)

```

00000010
00110010
01000000
00100000
00000001
00010011
01001000
00100010
00000010
10010101
01010000
00100100
00000001
00101010
01011000
00100101
00000001
01001011
01100000
00100110
00000010
00110010
01101000
00101010
00000000
00001100
10110000
10000010
00000000
00001101

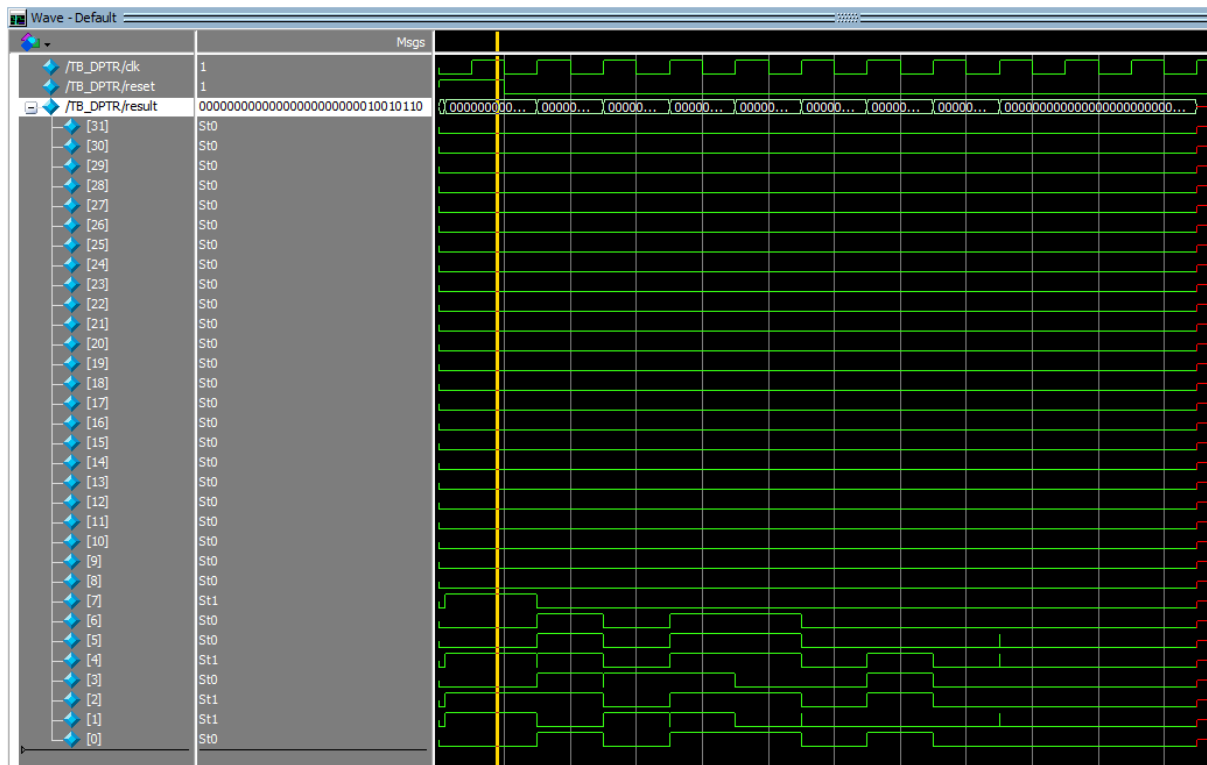
```

```
10111001
00000011
00000010
00110010
00000000
00110100
00000001
00001001
00000000
00110000
00000000
00000000
00000000
00000000
00100000
00001000
00000000
00001010
00101000
00001001
00000000
00001010
00110000
00001010
00000000
00001010
00110100
00001011
00000000
00001010
10101100
00001000
00000000
00000000
10001100
00001100
00000000
00000000
00010000
00000000
11111111
11111111
```

4.4 Resultados Simulación

Al correr la simulación, nuestro programa lee las instrucciones descritas en ensamblador (archivo .asm), las cuales fueron convertidas a binario mediante el script de python, devolviendonos un archivo .bin (se manda a llamar en el módulo Mem_I con readmemb) con las instrucciones en ceros y unos.

Luego con el módulo del test bench para el DPTR se le da valor a los registros utilizados en las instrucciones tipo R del ensamblador.



Estas son las señales obtenidas en la ventana Wave para el TB.

Memory Data - /TB_DPTR/tm/dp/BR/regs - Default		
0	0	0
2	0	0
4	0	0
6	0	0
8	150	125
10	10	127
12	117	0
14	0	0
16	0	100
18	50	25
20	30	10
22	29	0
24	0	0
26	0	0
28	0	0
30	0	0

Aquí puede observarse el memory list con los valores de cada registro después de correr la simulación, los valores corresponden a los resultados de cada operación.

Ahora nuestro procesador es capaz de entender las instrucciones tipo I, en la simulación se muestra como se guardan en la memoria

[illegible]

simulación en ModelSim donde se ven los datos tanto en binario como en decimal de todas las señales en general

Como se muestra a continuación, nuestro memory list funciona a la perfección:

◆ /B_DPTR/cm/zero_sig	0		
◆ /B_DPTR/cm/mem/...	2 50 64 32 1 19 7...		2 50 64
◆ [0]			2
◆ [1]	50		50
◆ [2]	64		64
◆ [3]	32		32
◆ [4]	1		1
◆ [5]	19		19
◆ [6]	72		72
◆ [7]	34		34
◆ [8]	2		2
◆ [9]	-107		-107
◆ [10]	80		80
◆ [11]	36		36
◆ [12]	1		1
◆ [13]	42		42
◆ [14]	88		88
◆ [15]	37		37
◆ [16]	1		1
◆ [17]	75		75
◆ [18]	96		96
◆ [19]	38		38
◆ [20]	2		2
◆ [21]	50		50
◆ [22]	104		104
◆ [23]	42		42
◆ [24]	0		0
◆ [25]	12		12
◆ [26]	-80		-80
◆ [27]	-126		-126
◆ [28]	0		0
◆ [29]	13		13
◆ [30]	-71		-71
◆ [31]	3		3
◆ [32]	2		2
◆ [33]	50		50

En el siguiente apartado se muestra cómo los buffers funcionan correctamente, donde se copia el valor para el siguiente bloque

00000000	00000004	00000008	0000000c	00000010
00000004	00000008	0000000c	00000010	00000014
02324020	01134822	02955024	012a5825	014b6026
00000000	02324020	01134822	02955024	012a5825

5. Conclusión

Al final de este proyecto, logramos que el procesador MIPS ejecutara correctamente las instrucciones tipo R desde un archivo binario. Fue una experiencia clave para entender que el hardware no solo requiere lógica, sino una coordinación perfecta entre el reloj, el PC y la memoria.

Aprendimos que pasar del ensamblador al binario mediante Python facilita mucho el trabajo, pero que el verdadero reto está en la simulación; ahí descubrimos que es vital inicializar cada registro y limpiar "la basura" de los datos para evitar errores en las señales. En conclusión, cumplimos el objetivo de integrar todos los módulos y dejamos una base sólida para implementar instrucciones de salto y memoria más adelante.

Comprendimos cómo funcionan las instrucciones tipo I que son las inmediatas y las instrucciones tipo J que son los saltos, para ello tuvimos que implementar un módulo que extendiera el signo donde utilizamos una sintaxis específica en verilog que replica 16 veces lo que hay en cierta posición del arreglo y lo pega en la instrucción.

Utilizamos la memoria de datos para pasar datos de ella misma al banco de registros con la instrucción lw load word y del banco de registros a guardarlo a la memoria con la función sw store word, con esto nuestro procesador ya está más completo y ahora podemos cargarle algoritmos un poco más complejos.

Como aportación adicional al diseño, implementamos el "pipelining" mediante la integración de Buffers de Aislamiento.

Comprendimos que estos registros están sincronizados por una señal de reloj en común, son cruciales para dividir el procesador en etapas independientes y que estos buffers dividen el procesador en 5 etapas las cuales son:

1. IF (Instruction Fetch): Búsqueda de instrucción.
2. ID (Instruction Decode): Decodificación y lectura de registros.
3. EX (Execute): Ejecución en la ALU.
4. MEM (Memory): Acceso a la Memoria de Datos.
5. WB (Write Back): Escritura en el Banco de Registros.

Las cuales fueron ejecutadas exitosamente como se mostraba en la simulación.

6. Bibliografía

- Wikipedia. (2025). MIPS architecture. Wikipedia. Recuperado de https://en.wikipedia.org/wiki/MIPS_architecture
- GeeksforGeeks. (2024). Instruction format in MIPS. Recuperado de <https://www.geeksforgeeks.org/instruction-format-in-mips/>
- GeeksforGeeks. (2024). Conditional or ternary operator in programming. Recuperado de <https://www.geeksforgeeks.org/conditional-or-ternary-operator-in-programming/>
- Patterson, D. A., & Hennessy, J. L. (2017). Computer organization and design MIPS edition: The hardware/software interface (5th ed.). Morgan Kaufmann.
- Mano, M. M., & Ciletti, M. D. (2017). Digital design (6th ed.). Pearson.
- TutorialsPoint. (2024). Compiler design phases. Recuperado de https://www.tutorialspoint.com/compiler_design/compiler_design_phases.htm
- Nandland. (2023). Verilog conditional operator. Recuperado de <https://nandland.com/conditional-operator/>
- MIPS Technologies, Inc. (2016). MIPS32 Architecture For Programmers Volume I-A: Introduction to the MIPS32 Architecture. <https://www.mips.com>
- Lenovo. (s. f.). ¿Qué es MIPS?. Glosario de Lenovo. Recuperado el 1 de mayo de 2026, de <https://www.lenovo.com/es/es/glossary/mips/>
- MIPS Assembly/Instruction Formats - Wikibooks, open books for an open world. (n.d.). https://en.wikibooks.org/wiki/MIPS_Assembly/Instruction_Formats
- De Nihil, V. T. L. E. (2019, September 9). Introducción a la Arquitectura del MIPS. La Cripta Del Hacker. <https://lacriptadelhacker.wordpress.com/2019/09/09/introduccion-a-la-arquitectura-de-l-mips/>
- colaboradores de Wikipedia. (2024, December 6). Arquitectura en pipeline (informática). Wikipedia, La Enciclopedia Libre. [https://es.wikipedia.org/wiki/Arquitectura_en_pipeline_\(inform%C3%A1tica\)](https://es.wikipedia.org/wiki/Arquitectura_en_pipeline_(inform%C3%A1tica))
- CICS Transaction Server for z/OS. (2026, January 16). <https://www.ibm.com/docs/es/cics-ts/6.x?topic=concepts-pipelining>
- Startup, E. E. (2026, April 13). CPU Pipelining: lo que todo CTO debe saber – El Ecosistema Startup. El Ecosistema Startup. <https://ecosistemastartup.com/cpu-pipelining-lo-que-todo-cto-debe-saber/>
-